TRƯỜNG ĐẠI HỌC CÔNG NGHỆ SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TIN HỌC

XÂY DỰNG CHƯƠNG TRÌNH QUẢN LÝ THƯ VIỆN BẰNG NGÔN NGỮ C++

(Ứng dụng các Cấu trúc Dữ liệu & Giải thuật, lưu trữ trên file, không dùng CSDL)

TÀI LIỆU THIẾT KẾ (DESIGN) – TUẦN 2

Giảng viên hướng dẫn: Lương An Vinh
Sinh viên thực hiện: Trọng Nghĩa
MSSV: DH52400659
Lớp: DH24_TH04
GitHub: https://github.com/huynhnghia14/LibraryManagement

Học kỳ I, năm học 2026 – 2027
TP. Hồ Chí Minh, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU.....5
1.1. Mục đích.....5
1.2. Phạm vi và tài liệu liên quan.....5
1.3. Nguyên tắc thiết kế.....5
CHƯƠNG 2. KIẾN TRÚC TỔNG THỂ VÀ SƠ ĐỒ MODULE.....6
2.1. Kiến trúc phân tầng.....6
2.2. Sơ đồ module (file .h / .cpp).....7
2.3. Cấu trúc thư mục và danh sách file.....7
CHƯƠNG 3. THIẾT KẾ DỮ LIỆU – ĐỊNH NGHĨA STRUCT.....9
3.1. Sơ đồ các struct dữ liệu.....9
3.2. Kiểu Date.....10
3.3. Struct Book, Reader, BorrowTicket, Transaction, Account.....10
3.4. Các kiểu phụ trợ.....11
CHƯƠNG 4. THIẾT KẾ CẤU TRÚC DỮ LIỆU CHO TỪNG CHỨC NĂNG.....12
4.1. Sơ đồ lớp các CTDL.....12
4.2. Cây AVL – quản lý sách theo ISBN.....13
4.3. Cây BST – danh mục sách theo tên.....14
4.4. Bảng băm – quản lý độc giả theo CMND.....15
4.5. Danh sách liên kết đơn – phiếu mượn.....16
4.6. Mảng động – lịch sử giao dịch.....16
4.7. Đồ thị danh sách kề – gợi ý sách.....17
4.8. Cây Trie – autocomplete tên sách (CTDL tự tìm hiểu).....18
CHƯƠNG 5. THIẾT KẾ LỚP LIBRARY, SERVICE VÀ GIAO DIỆN.....20
5.1. Sơ đồ lớp ứng dụng.....20
5.2. Lớp Library (facade).....20
5.3. Các lớp Service.....21
5.4. Lớp Menu, UI, Auth.....21

CHƯƠNG 6. THIẾT KẾ LƯU TRỮ FILE.....23
6.1. Định dạng file nhị phân.....23
6.2. Luồng nạp / lưu / sao lưu.....24
CHƯƠNG 7. THIẾT KẾ MENU VÀ LUỒNG CHƯƠNG TRÌNH.....25
7.1. Cây menu.....25
7.2. Luồng chính của chương trình.....26
7.3. Luồng lập phiếu mượn.....27
7.4. Luồng trả sách và tính phạt.....28
7.5. Luồng thêm sách.....29
7.6. Luồng gợi ý sách.....30
7.7. Phác thảo màn hình.....30
CHƯƠNG 8. THIẾT KẾ GIẢI THUẬT CHÍNH (MÃ GIẢ).....31
8.1. Kiểm tra ràng buộc và lập phiếu mượn.....31
8.2. Trả sách và tính phạt.....31
8.3. Top 10 sách / độc giả.....32
8.4. Gợi ý sách cùng thể loại (BFS).....32
8.5. Autocomplete bằng Trie.....32
8.6. Chuẩn hóa chuỗi tiếng Việt.....32
CHƯƠNG 9. THIẾT KẾ XỬ LÝ LỖI VÀ KIỂM THỬ.....33
9.1. Xử lý lỗi và kiểm tra dữ liệu nhập.....33
9.2. Kế hoạch kiểm thử.....33
CHƯƠNG 10. KẾ HOẠCH CÀI ĐẶT THEO THIẾT KẾ.....35
10.1. Các điểm cần giảng viên góp ý tại mốc #1.....35
PHỤ LỤC.....36
PL.1. Ma trận truy vết thiết kế – yêu cầu.....36
PL.2. Quy ước đặt tên và mã nguồn.....36

DANH MỤC HÌNH

Hình 2.1. Kiến trúc phân tầng của chương trình.....6

Hình 2.2. Sơ đồ phụ thuộc giữa các module (mũi tên: #include / sử dụng).....7

Hình 3.1. Các struct dữ liệu (POD) và quan hệ tham chiếu qua khóa.....9

Hình 4.1. Sơ đồ lớp các cấu trúc dữ liệu tự cài đặt.....12

Hình 4.2. Ví dụ mất cân bằng Trái-Trái khi chèn ISBN và phép quay phải khôi phục cân bằng.....13

Hình 4.3. Ví dụ cây BST theo tên chuẩn hóa và thứ tự duyệt in-order.....14

Hình 4.4. Bảng băm với xử lý va chạm separate chaining.....15

Hình 4.5. Danh sách liên kết đơn các phiếu mượn.....16

Hình 4.6. Mảng động tăng gấp đôi dung lượng khi đầy.....16

Hình 4.7. Đồ thị sách với hai loại cạnh và biểu diễn danh sách kề.....17

Hình 4.8. Ví dụ Trie chứa tên sách chuẩn hóa và truy vấn tiền tố "lap".....18

Hình 5.1. Sơ đồ lớp tầng ứng dụng: Library (facade), các Service, Menu và lưu trữ. .20

Hình 6.1. Bố cục file nhị phân: header + mảng bản ghi kích thước cố định.....23

Hình 6.2. Luồng nạp dữ liệu khi khởi động, lưu khi thay đổi và sao lưu khi thoát.....24

Hình 7.1. Cây menu chính và các menu con.....25

Hình 7.2. Lưu đồ luồng chính: khởi động → đăng nhập → vòng lặp menu → lưu & thoát.....26

Hình 7.3. Lưu đồ lập phiếu mượn với 6 ràng buộc (a)–(f) theo FR-L-03.....27

Hình 7.4. Lưu đồ trả sách và tính tiền phạt quá hạn.....28

Hình 7.5. Lưu đồ thêm sách: cập nhật đồng thời AVL, BST, Trie và đồ thị.....29

Hình 7.6. Lưu đồ gợi ý sách cùng thể loại (BFS) và theo lịch sử mượn.....30

CHƯƠNG 1. GIỚI THIỆU

1.1. Mục đích

Tài liệu này trình bày thiết kế chi tiết của chương trình Quản lý thư viện bằng C++ dựa trên các yêu cầu đã đặc tả trong tài liệu SRS (tuần 1). Nội dung gồm: kiến trúc tổng thể và sơ đồ module; định nghĩa các struct dữ liệu; thiết kế từng cấu trúc dữ liệu (CTDL) gắn với chức năng; sơ đồ lớp; thiết kế lưu trữ file; thiết kế menu và luồng chương trình; giải thuật chính dưới dạng mã giả; thiết kế xử lý lỗi và kiểm thử.

Tài liệu là sản phẩm nộp của tuần 2 và là căn cứ để giảng viên duyệt thiết kế tại mốc kiểm tra #1. Khi cài đặt (tuần 3 – 8), mọi thay đổi so với thiết kế sẽ được ghi nhận và cập nhật lại tài liệu để đưa vào Chương 2 của báo cáo cuối kỳ.

1.2. Phạm vi và tài liệu liên quan

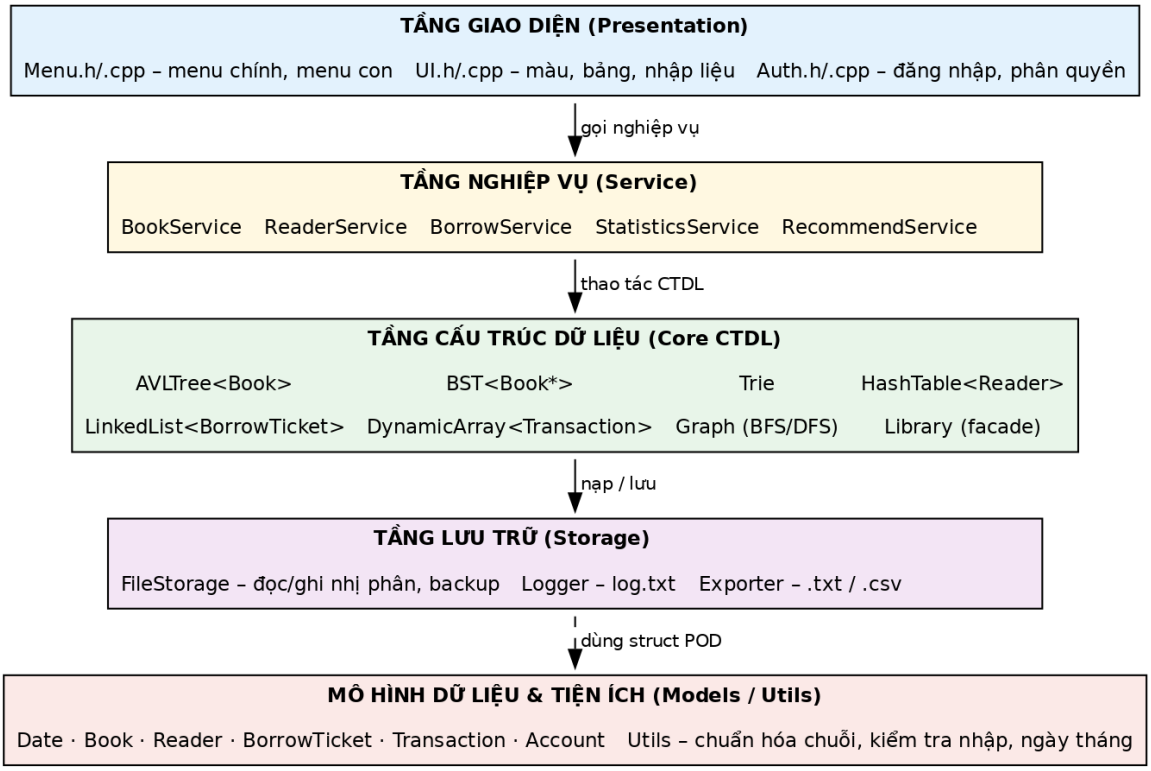
- **SRS.pdf** – Đặc tả yêu cầu phần mềm (tuần 1): các mã yêu cầu FR-xx, quy tắc nghiệp vụ BR-xx được tham chiếu trực tiếp trong tài liệu này.
- **Đề bài** – Đồ án Tin học HK I (2026-2027), GV Lương An Vinh: mục 4 (CTDL bắt buộc), mục 5 (CTDL tự tìm hiểu), mục 6 (yêu cầu lập trình).
- **README.md** trong repo GitHub: hướng dẫn biên dịch, cấu trúc thư mục.

1.3. Nguyên tắc thiết kế

- **Tách tầng:** giao diện (menu) – nghiệp vụ (service) – CTDL – lưu trữ; tầng trên chỉ gọi tầng ngay dưới.
- **Tự cài đặt CTDL:** toàn bộ AVL, BST, Hash Table, Linked List, Dynamic Array, Graph, Trie viết bằng template C++ với con trỏ thô, không dùng `std::map`, `std::set`, `std::unordered_map`, `std::list`, `std::vector` thay thế.
- **Struct POD cho dữ liệu:** mọi bản ghi dùng mảng `char` kích thước cố định và kiểu số nguyên để ghi / đọc file nhị phân trực tiếp bằng `fwrite`/`fread`.
- **Một nguồn sự thật:** dữ liệu sách lưu trong cây AVL; BST và Trie chỉ giữ con trỏ / chỉ số tham chiếu tới bản ghi trong AVL (index phụ), tránh trùng lặp dữ liệu.
- **Xử lý lỗi bằng giá trị trả về:** các hàm nghiệp vụ trả về `Result {bool ok; std::string message;}` để menu hiển thị thông báo; không dùng `exit()` giữa chừng.
- **Không goto, hạn chế biến toàn cục:** trạng thái duy nhất của chương trình nằm trong đối tượng `Library` được truyền tham chiếu.

CHƯƠNG 2. KIẾN TRÚC TỔNG THỂ VÀ SƠ ĐỒ MODULE

2.1. Kiến trúc phân tầng

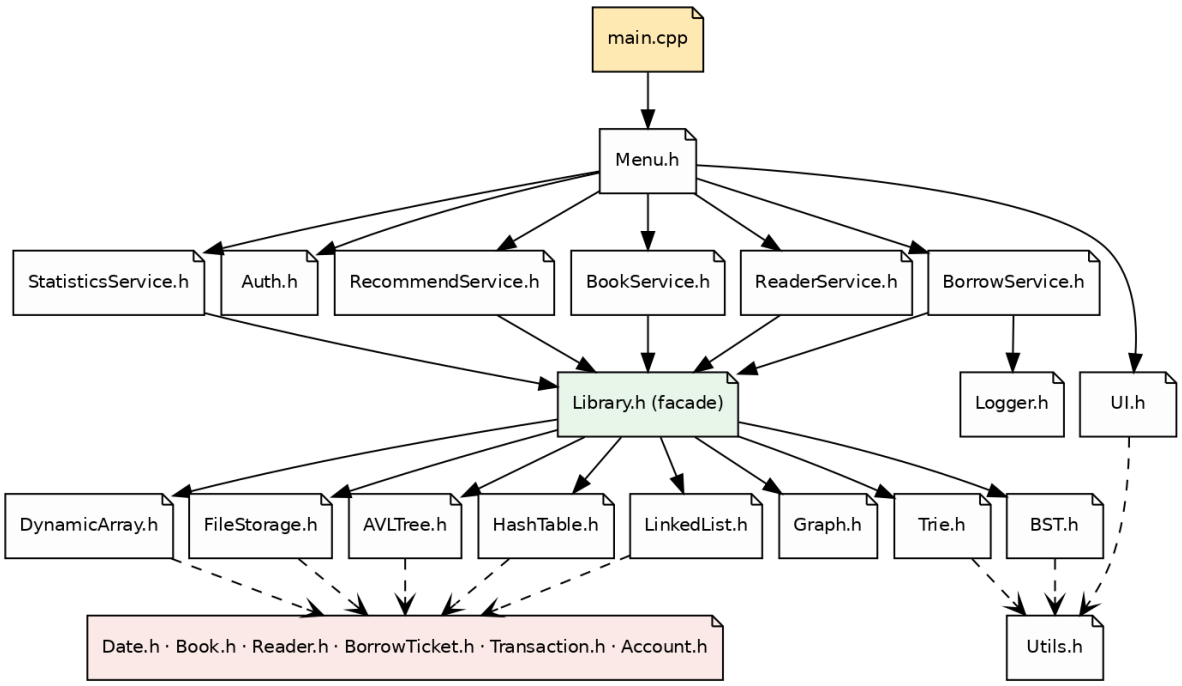


Hình 2.1. Kiến trúc phân tầng của chương trình

Bảng 2.1. Trách nhiệm từng tầng

Tầng	Thành phần	Trách nhiệm
Giao diện	Menu, UI, Auth	Hiển thị menu chính / menu con, nhận và kiểm tra dữ liệu nhập, in bảng có màu, đăng nhập và kiểm tra quyền. Không chứa logic nghiệp vụ.
Nghiệp vụ	BookService, ReaderService, BorrowService, StatisticsService, RecommendService	Thực hiện các use case: kiểm tra ràng buộc nghiệp vụ (BR-xx), phối hợp nhiều CTDL, ghi log, đánh dấu dữ liệu thay đổi (dirty).
CTDL	AVLTree, BST, HashTable, LinkedList, DynamicArray, Graph, Trie, Library	Các CTDL tổng quát (template) và lớp `Library` gom toàn bộ CTDL thành một đối tượng duy nhất (facade).
Lưu trữ	FileStorage, Logger, Exporter	Đọc / ghi file nhị phân có header, sao lưu – phục hồi, ghi nhật ký, xuất báo cáo văn bản / CSV.
Mô hình & tiện ích	Date, Book, Reader, BorrowTicket, Transaction, Account, Utils	Định nghĩa struct POD; hàm chuẩn hóa chuỗi (bỏ dấu, chữ thường), kiểm tra định dạng, xử lý ngày tháng.

2.2. Sơ đồ module (file .h / .cpp)



Hình 2.2. Sơ đồ phụ thuộc giữa các module (mũi tên: #include / sử dụng)

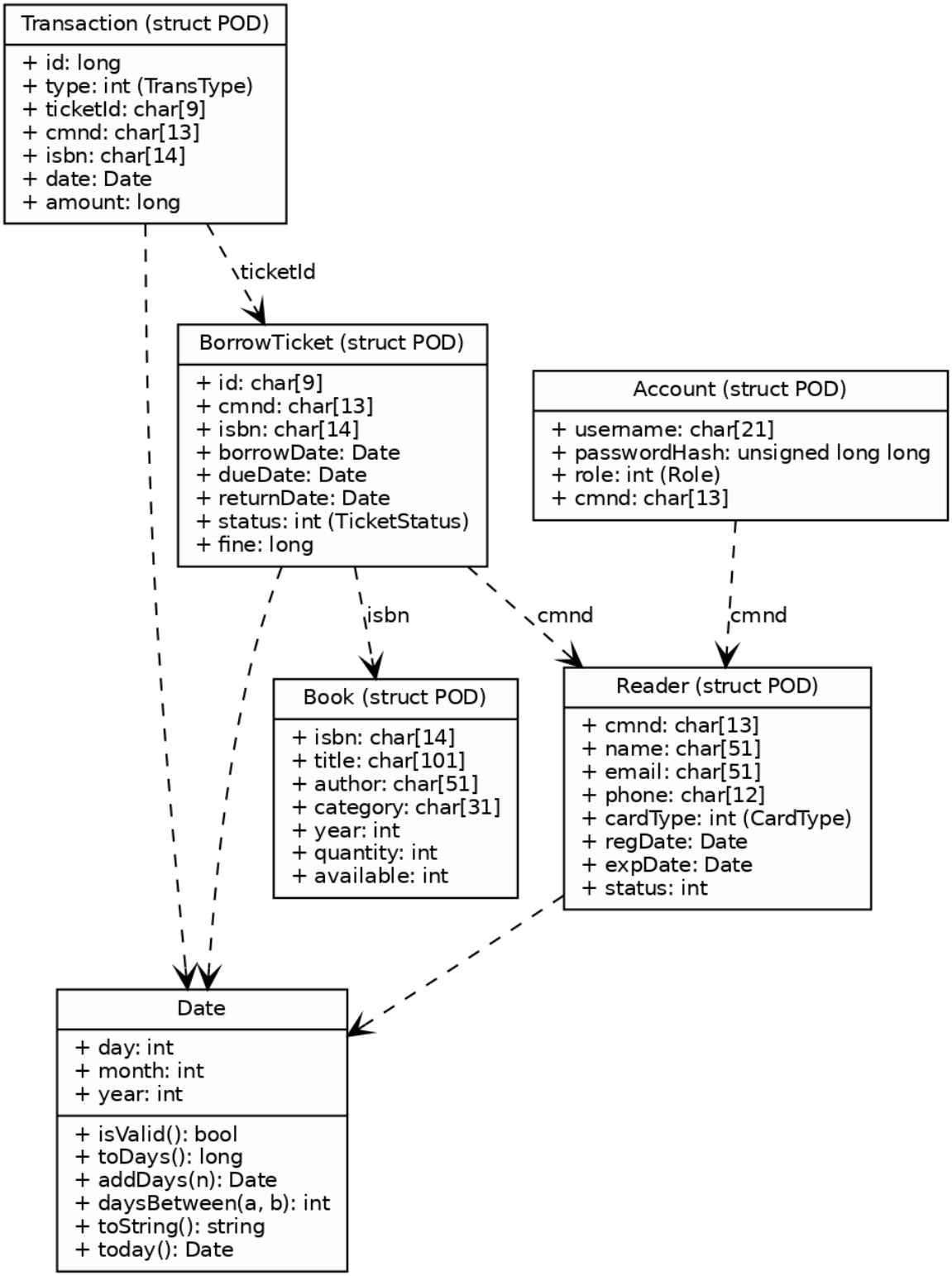
2.3. Cấu trúc thư mục và danh sách file

```
LibraryManagement/
    ├── include/
    │   ├── Date.h  Book.h  Reader.h  BorrowTicket.h  Transaction.h  Account.h
(struct POD + enum)
    │   ├── AVLTree.h  BST.h  HashTable.h  LinkedList.h  DynamicArray.h
(template, header-only)
    │   ├── Graph.h  Trie.h
(khai báo lớp)
    │   ├── Library.h  FileStorage.h  Logger.h  Exporter.h  Config.h
    │   ├── BookService.h  ReaderService.h  BorrowService.h  StatisticsService.h
RecommendService.h
    │   ├── Menu.h  UI.h  Auth.h  Utils.h  Result.h
    ├── src/
    │   ├── main.cpp
    │   ├── Date.cpp  Graph.cpp  Trie.cpp  Library.cpp  FileStorage.cpp
Logger.cpp  Exporter.cpp  Config.cpp
    │   ├── BookService.cpp  ReaderService.cpp  BorrowService.cpp
StatisticsService.cpp  RecommendService.cpp
    │   ├── Menu.cpp  MenuBook.cpp  MenuReader.cpp  MenuBorrow.cpp  MenuStats.cpp
MenuSystem.cpp
    │   └── UI.cpp  Auth.cpp  Utils.cpp
    ├── data/   (books.dat, readers.dat, tickets.dat, history.dat, accounts.dat,
config.txt, log.txt, sample/, reports/, backup/)
    ├── docs/   (SRS.pdf, Design.pdf, images/, src/)
    ├── tests/  (test_avl.cpp  test_bst.cpp  test_hash.cpp  test_linkedlist.cpp
test_dynarray.cpp
    │            test_graph.cpp  test_trie.cpp  test_date.cpp  test_borrow.cpp
benchmark.cpp)
    ├── Makefile
    └── README.md
```

Các CTDL template (AVLTree, BST, HashTable, LinkedList, DynamicArray) đặt toàn bộ trong file .h vì C++ yêu cầu định nghĩa template có mặt tại nơi sử dụng. Graph và Trie không phải template nên tách .h / .cpp. Menu được chia thành nhiều file .cpp theo từng nhóm chức năng để mỗi file dưới 300 dòng.

CHƯƠNG 3. THIẾT KẾ DỮ LIỆU – ĐỊNH NGHĨA STRUCT

3.1. Sơ đồ các struct dữ liệu



Hình 3.1. Các struct dữ liệu (POD) và quan hệ tham chiếu qua khóa

3.2. Kiểu Date

```
// include/Date.h
    struct Date {
        int day, month, year;
        // 1..31, 1..12, 1900..2100, kiểm tra năm nhuận
        bool isValid() const;
        // số ngày kể từ 01/01/1900 (để trừ ngày)
        long toDays() const;
        static Date fromDays(long days);
        Date addDays(int n) const;                  // ngày hẹn trả = ngày mượn +
n
        static int daysBetween(const Date& a, const Date& b); // b - a
        static Date today();                        // lấy từ <ctime>
        std::string toString() const;               // "dd/mm/yyyy"
        static bool parse(const std::string& s, Date& out);
        bool operator<(const Date& o) const; bool operator==(const Date& o) const;
    };
```

3.3. Struct Book, Reader, BorrowTicket, Transaction, Account

```
// include/Book.h
    const int ISBN_LEN = 13, TITLE_LEN = 100, AUTHOR_LEN = 50, CATEGORY_LEN = 30;
    struct Book {
        char isbn[ISBN_LEN + 1];      // khóa chính, 13 chữ số, kết thúc '\0'
        char title[TITLE_LEN + 1];
        char author[AUTHOR_LEN + 1];
        char category[CATEGORY_LEN + 1];
        int  year;                    // 1900..năm hiện tại
        int  quantity;                // tổng số bản
        int  available;               // số bản còn trong kho (0..quantity)
    };

    // include/Reader.h
    enum CardType   { CARD_STUDENT = 0, CARD_LECTURER = 1, CARD_NORMAL = 2 };
    enum ReaderStatus { READER_ACTIVE = 0, READER_LOCKED = 1 };
    struct Reader {
        char cmnd[13];                // khóa chính, 9 hoặc 12 chữ số
        char name[51];
        char email[51];
        char phone[12];               // 10 chữ số
        int  cardType;                // CardType
        Date regDate, expDate;        // expDate = regDate + 12 tháng
        int  status;                  // ReaderStatus
    };

    // include/BorrowTicket.h
    enum TicketStatus { TICKET_BORROWING = 0, TICKET_RETURNED = 1, TICKET_OVERDUE
= 2 };
    struct BorrowTicket {
        char id[9];                   // "PM00001"
        char cmnd[13];                // -> Reader
        char isbn[ISBN_LEN + 1];      // -> Book
        Date borrowDate, dueDate, returnDate;   // returnDate = {0,0,0} nếu chưa
trả
        int  status;                  // TicketStatus
        long fine;                    // tiền phạt (đồng), tính khi trả
    };

    // include/Transaction.h
    enum TransType { TRANS_BORROW = 0, TRANS_RETURN = 1, TRANS_FINE = 2 };
    struct Transaction {
        long id;                      // tự tăng
        int  type;                    // TransType
        char ticketId[9];
        char cmnd[13];
        char isbn[ISBN_LEN + 1];
        Date date;
```

```
        long amount;                 // tiền phạt với TRANS_FINE, 0 với loại khác
    };

    // include/Account.h
    enum Role { ROLE_ADMIN = 0, ROLE_LIBRARIAN = 1, ROLE_READER = 2 };
    struct Account {
        char username[21];
        unsigned long long passwordHash;   // FNV-1a(salt + password)
        int  role;                   // Role
        char cmnd[13];               // liên kết độc giả (rỗng với Admin / Thủ
thư)
    };
```

Bảng 3.1. Ước tính kích thước bản ghi và file

Struct	Kích thước ước tính (byte)	Số bản ghi dự kiến	Dung lượng file
Book	14 + 101 + 51 + 31 + 3×4 ≈ 212 (có padding)	50 – 10.000	≈ 2 MB với 10.000 sách
Reader	13 + 51 + 51 + 12 + 4 + 2×12 + 4 ≈ 160	20 – 2.000	≈ 320 KB
BorrowTicket	9 + 13 + 14 + 3×12 + 4 + 8 ≈ 88	30 – 50.000	≈ 4,4 MB
Transaction	8 + 4 + 9 + 13 + 14 + 12 + 8 ≈ 72	60 – 100.000	≈ 7,2 MB
Account	21 + 8 + 4 + 13 ≈ 48	3 – 2.000	≈ 96 KB

Tất cả struct chỉ chứa kiểu cơ bản và mảng `char` nên thỏa điều kiện POD / trivially copyable: có thể ghi bằng `fwrite(&b, sizeof(Book), 1, f)`. Hằng số độ dài khai báo tập trung trong `Book.h` / `Utils.h`; khi đổi độ dài phải tăng `version` trong header file để phát hiện file cũ.

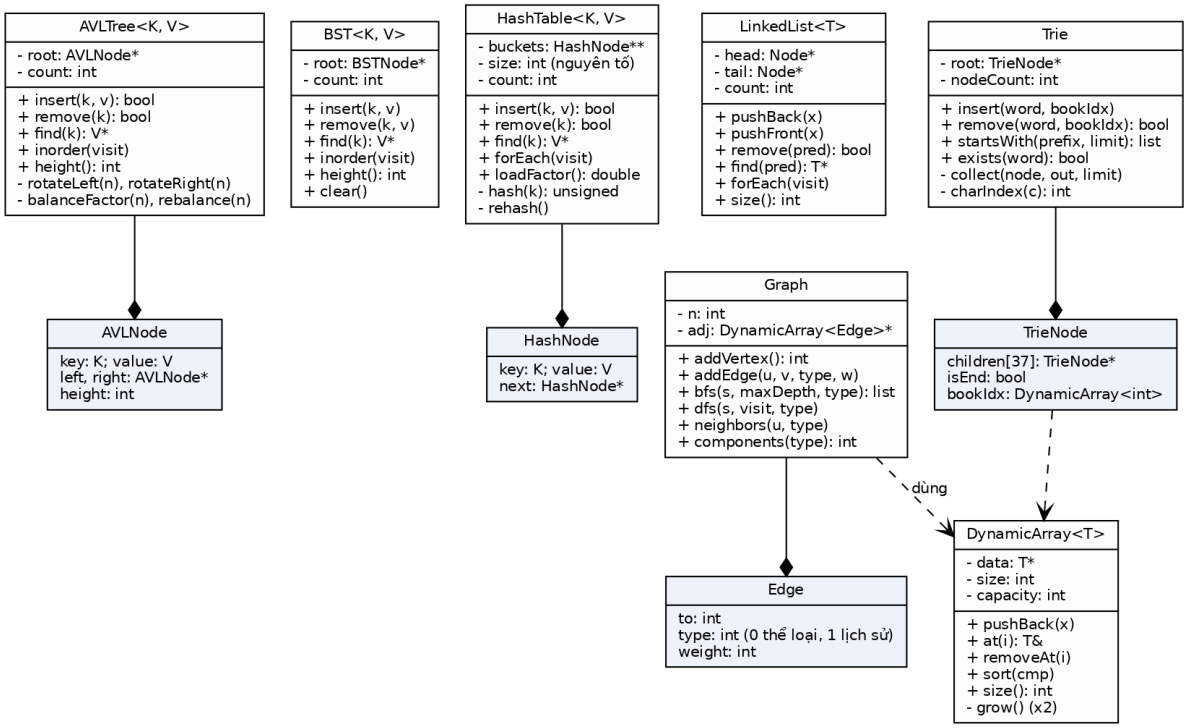
3.4. Các kiểu phụ trợ

```
// include/Result.h
    struct Result { bool ok; std::string message; static Result success(const
std::string& m = ""); static Result fail(const std::string& m); };

    // include/Config.h  (đọc từ data/config.txt, có giá trị mặc định)
    struct Config {
        long finePerDay = 2000;                  // đồng / ngày / cuốn
        int  maxBooks[3]   = {3, 5, 2};          // theo CardType: SV, GV, Thường
        int  loanDays[3]   = {14, 30, 7};
        int  cardValidMonths = 12;
        int  backupKeep = 30;
        bool useColor = true; std::string lang = "vi";
    };
```

CHƯƠNG 4. THIẾT KẾ CẤU TRÚC DỮ LIỆU CHO TỪNG CHỨC NĂNG

4.1. Sơ đồ lớp các CTDL



Hình 4.1. Sơ đồ lớp các cấu trúc dữ liệu tự cài đặt

Bảng 4.1. Ánh xạ CTDL – chức năng – thao tác – độ phức tạp

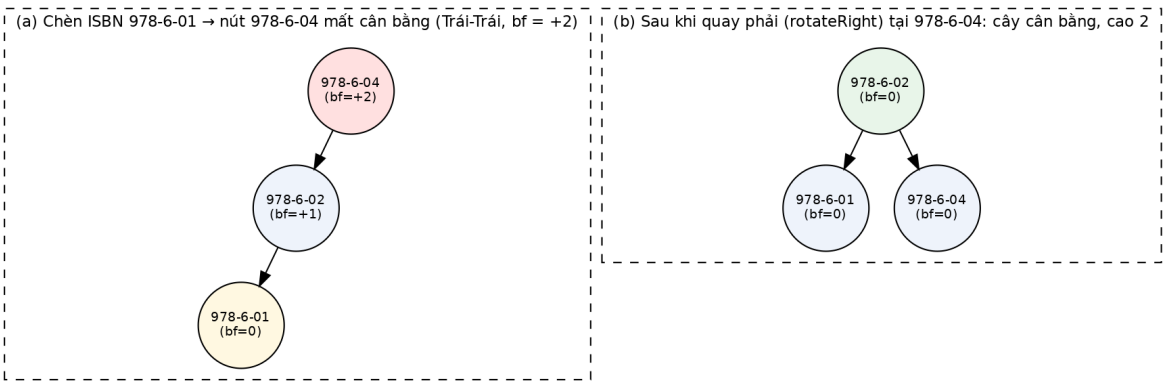
CTDL	Khóa / phần tử	Chức năng (mã FR)	Thao tác chính	Độ phức tạp
AVLTree<string, Book>	ISBN → Book	FR-B-01..05, FR-L-01, FR-S-02, FR-G-01	insert, remove, find, inorder	O(log n) / O(log n) / O(log n) / O(n)
BST<string, Book*>	tên chuẩn hóa → Book*	FR-B-06	insert, remove, inorder	TB O(log n), xấu nhất O(n)
Trie	tên chuẩn hóa → danh sách chỉ số sách	FR-B-07, FR-G-03, FR-G-04	insert, remove, startsWith	O(L) / O(L) / O(|p| + k)
HashTable<string, Reader>	CMND → Reader	FR-R-01..06, FR-L-01, FR-S-03	insert, remove, find, forEach	O(1) trung bình, O(n) xấu nhất
LinkedList<BorrowTicket>	phiếu mượn	FR-L-01..05	pushBack, find(pred), remove(pred), forEach	O(1) / O(m) / O(m) / O(m)

CTDL	Khóa / phần tử	Chức năng (mã FR)	Thao tác chính	Độ phức tạp
DynamicArray<Transaction>	giao dịch	FR-L-06, FR-S-01..07, FR-G-02	pushBack, at, sort	O(1) khấu hao / O(1) / O(n log n)
Graph	đỉnh = sách, cạnh loại 0/1	FR-G-01, FR-G-02, FR-G-05	addEdge, bfs, dfs, neighbors	O(1) / O(V+E) / O(V+E) / O(deg)
HashTable<string, Account>	username → Account	FR-X-01	find, insert	O(1) trung bình

4.2. Cây AVL – quản lý sách theo ISBN

Vai trò: kho chứa chính của toàn bộ sách; khóa là chuỗi ISBN 13 chữ số (so sánh chuỗi tương đương so sánh số vì độ dài cố định). Mọi CTDL khác liên quan đến sách (BST, Trie, Graph) chỉ lưu con trỏ `Book*` hoặc chỉ số trỏ về nút AVL, do đó khi sửa thông tin sách chỉ cần sửa một nơi.

```
template <typename K, typename V>
    class AVLTree {
        struct Node { K key; V value; Node* left; Node* right; int height; };
        Node* root; int count;
        int  h(Node* n) const { return n ? n->height : 0; }
        int  balanceFactor(Node* n) const { return h(n->left) - h(n->right); }
        Node* rotateLeft(Node* x);  Node* rotateRight(Node* y);
        Node* rebalance(Node* n);                    // 4 trường hợp LL, RR, LR,
RL
        Node* insertRec(Node* n, const K& k, const V& v, bool& inserted);
        // thay bằng nút nhỏ nhất bên phải
        Node* removeRec(Node* n, const K& k, bool& removed);
        Node* minNode(Node* n);
    public:
        bool insert(const K& k, const V& v);         // false nếu khóa đã tồn tại
        bool remove(const K& k);
        V*   find(const K& k);                       // nullptr nếu không có
        template <typename F> void inorder(F visit); // duyệt tăng dần theo khóa
        int  size() const; int height() const; void clear();
    };
```



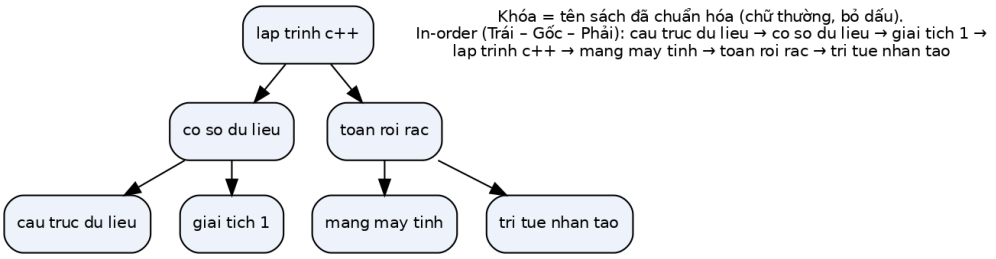
Hình 4.2. Ví dụ mất cân bằng Trái-Trái khi chèn ISBN và phép quay phải khôi phục cân bằng

Bất biến: |balanceFactor| ≤ 1 tại mọi nút; chiều cao ≤ $1{,}44 \cdot \log_2(n+2)$. Sau mỗi `insertRec`/`removeRec` cập nhật `height` và gọi `rebalance` trên đường đi về gốc. Unit test kiểm tra bất biến sau mỗi thao tác bằng hàm `checkBalanced()`.

Lý do chọn AVL thay vì BST cho ISBN: sách thường được nhập theo lô với ISBN tăng dần (cùng nhà xuất bản) → BST suy biến thành danh sách, tìm kiếm O(n). AVL giữ O(log n) trong mọi thứ tự nhập. Kết quả đo tuần 9 sẽ minh họa điều này.

4.3. Cây BST – danh mục sách theo tên

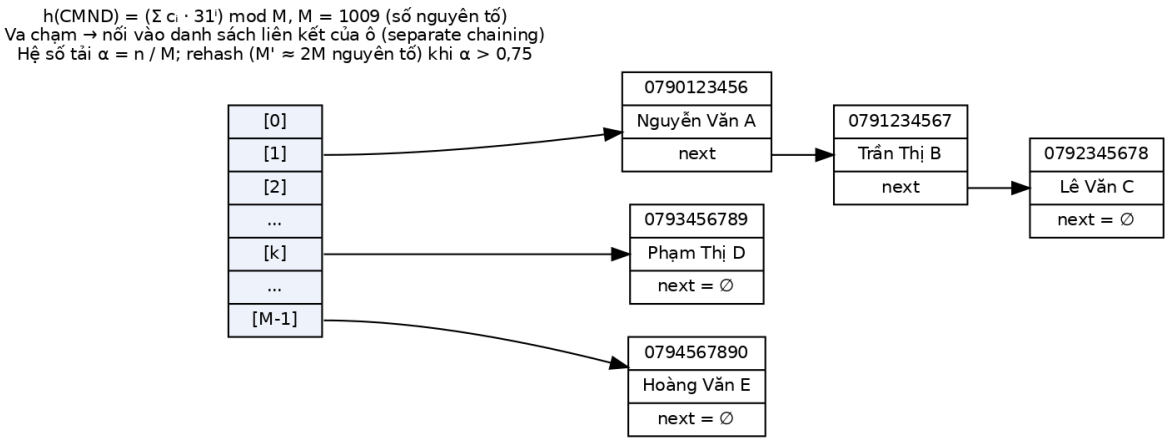
Vai trò: chỉ mục phụ theo tên để in danh mục sắp theo thứ tự chữ cái bằng duyệt in-order (FR-B-06), đồng thời là CTDL "đã học" để so sánh với AVL (hiệu năng) và với Trie (tìm theo tiền tố). Khóa là tên chuẩn hóa (`Utils::normalize`: chữ thường, bỏ dấu tiếng Việt, gộp khoảng trắng); giá trị là `Book*`. Hai sách trùng tên được chấp nhận: khóa bằng nhau đi sang nhánh phải, và `remove(k, v)` xóa đúng nút có con trỏ `v`.



Hình 4.3. Ví dụ cây BST theo tên chuẩn hóa và thứ tự duyệt in-order

```
template <typename K, typename V>
    class BST {
        struct Node { K key; V value; Node *left, *right; };
        Node* root; int count;
    public:
        // lặp (không đệ quy) để tránh tràn stack khi cây lệch
        void insert(const K& k, const V& v);
        // xóa nút có khóa k và value v (3 trường hợp: lá, 1 con, 2 con)
        bool remove(const K& k, const V& v);
        V*   find(const K& k);
        template <typename F> void inorder(F visit);
        // dùng cho so sánh với Trie
        template <typename F> void prefixSearch(const K& prefix, F visit);
        int  height() const; int size() const; void clear();
    };
```

4.4. Bảng băm – quản lý độc giả theo CMND

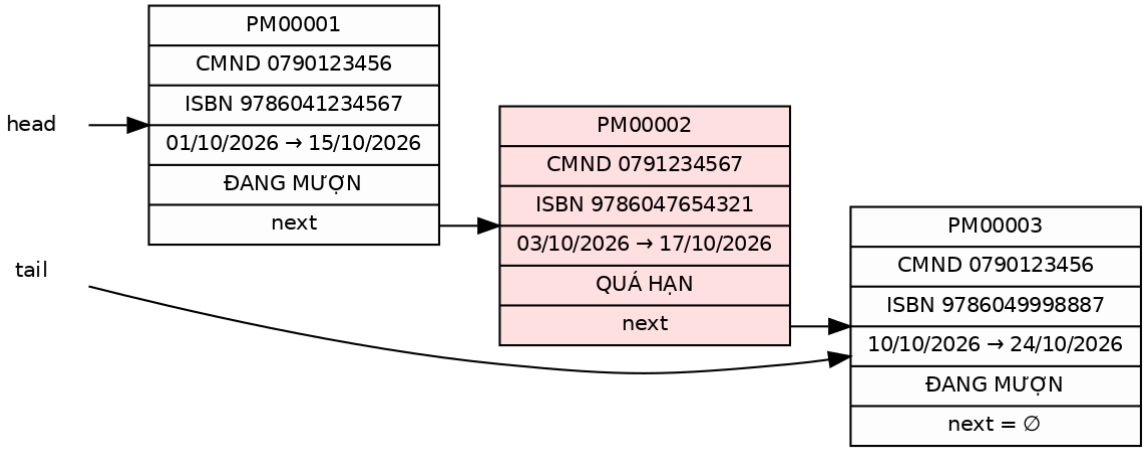


Hình 4.4. Bảng băm với xử lý va chạm separate chaining

```
template <typename K, typename V>
    class HashTable {
        struct Node { K key; V value; Node* next; };
        // size là số nguyên tố (1009, 2003, 4007, ...)
        Node** buckets; int size; int count;
        unsigned hash(const K& key) const;           // đa thức: h = (h * 31 + c)
% size
        // khi loadFactor() > 0.75: size mới ≈ 2*size (nguyên tố), chèn lại
        void rehash();
    public:
        explicit HashTable(int initialSize = 1009);
        bool insert(const K& k, const V& v);         // false nếu trùng khóa
        bool remove(const K& k);
        V*   find(const K& k);
        template <typename F> void forEach(F visit); // duyệt mọi bucket (thống
kê, in danh sách)
        double loadFactor() const { return (double)count / size; }
        // phục vụ đo hiệu năng
        int  size() const; int bucketCount() const; int longestChain() const;
    };
```

Hàm băm: CMND là chuỗi 9–12 chữ số; hàm băm đa thức cơ số 31 lấy dư theo kích thước bảng (số nguyên tố) phân bố tốt với các chuỗi số gần nhau. **Va chạm:** nối vào đầu danh sách của bucket (O(1)). Với 2.000 độc giả và bảng 2003 ô, hệ số tải $\alpha \approx 1$, chuỗi trung bình ~1 nút. Lý do chọn bảng băm: tra cứu độc giả là thao tác chính xác theo khóa (không cần sắp thứ tự) → O(1) trung bình, tốt hơn cây O(log n); thống kê theo loại thẻ chỉ cần duyệt toàn bộ.

4.5. Danh sách liên kết đơn – phiếu mượn



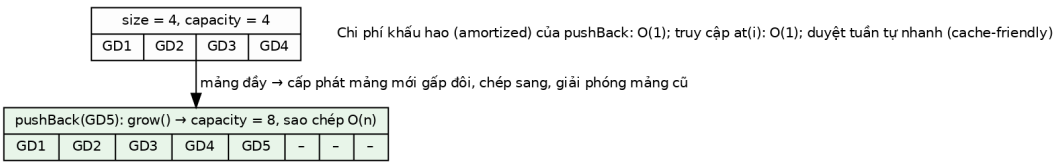
Hình 4.5. Danh sách liên kết đơn các phiếu mượn

```
template <typename T>
    class LinkedList {
        struct Node { T data; Node* next; };
        Node *head, *tail; int count;
    public:
        void pushBack(const T& x);                  // O(1) nhờ con trỏ tail
        // trả con trỏ phần tử đầu tiên thỏa pred
        template <typename P> T*   find(P pred);
        template <typename P> bool remove(P pred);   // xóa phần tử đầu tiên thỏa
pred, O(m)
        template <typename F> void forEach(F visit);
        template <typename P> int  countIf(P pred);  // đếm phiếu đang mượn của
một độc giả
        int size() const; void clear();
    };
```

Thiết kế dữ liệu phiếu: danh sách chứa *mọi* phiếu (đang mượn, quá hạn, đã trả) để in lịch sử phiếu theo độc giả; các truy vấn "đang mượn" dùng `countIf`/`find` với vị từ theo `status`. Khi tải từ file, phiếu được thêm theo thứ tự thời gian nên danh sách tự sắp theo mã phiếu tăng dần. Mã phiếu kế tiếp = số phiếu lớn nhất + 1 (tính khi nạp).

Lý do: phiếu được thêm vào cuối và duyệt tuần tự; số phiếu đang mượn nhỏ (≤ số độc giả × giới hạn) nên tìm O(m) chấp nhận được; danh sách liên kết không cần cấp phát lại như mảng.

4.6. Mảng động – lịch sử giao dịch



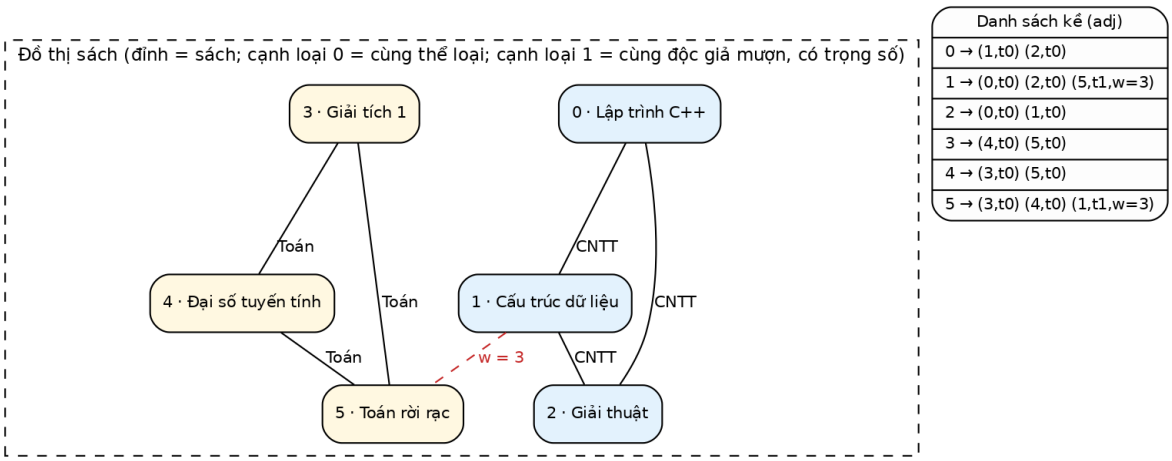
Hình 4.6. Mảng động tăng gấp đôi dung lượng khi đầy

```
template <typename T>
    class DynamicArray {
```

```
        T* data; int size_, capacity_;
        void grow() { reserve(capacity_ == 0 ? 8 : capacity_ * 2); }
    public:
        void pushBack(const T& x);                    // O(1) khấu hao
        T&   at(int i); const T& at(int i) const;     // kiểm tra biên, ném
std::out_of_range
        void removeAt(int i);                         // dịch trái O(n) (hiếm dùng)
        void reserve(int cap);                        // dùng khi đọc file: biết
trước count
        // sắp xếp (merge sort tự cài) cho top 10
        template <typename C> void sort(C cmp);
        int  size() const; T* raw();                  // raw() để fwrite cả khối
        void clear();
    };
```

Lý do: lịch sử chỉ thêm (append-only) và được quét toàn bộ khi thống kê → mảng liên tục cho truy cập O(1), duyệt nhanh và ghi file một lần bằng `fwrite(raw(), sizeof(T), size, f)`. Khi đọc file gọi `reserve(count)` để tránh cấp phát lại nhiều lần.

4.7. Đồ thị danh sách kề – gợi ý sách



Hình 4.7. Đồ thị sách với hai loại cạnh và biểu diễn danh sách kề

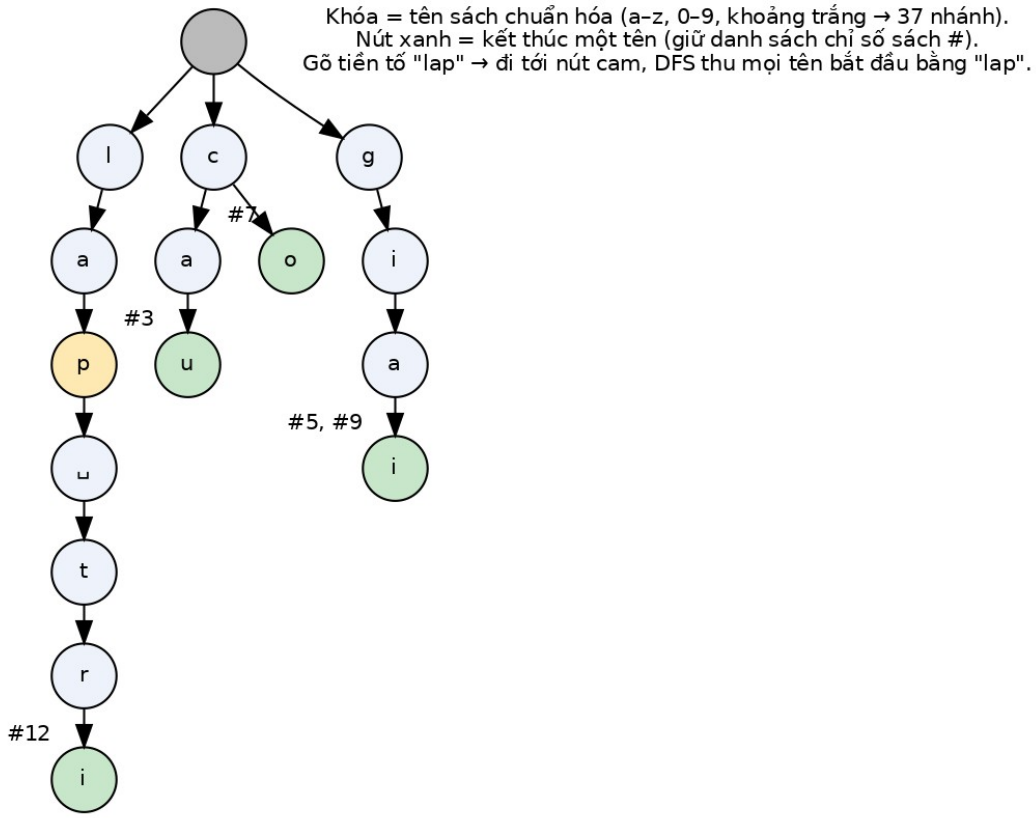
```
// type: 0 = cùng thể loại, 1 = cùng độc giả mượn
struct Edge { int to; int type; int weight; };
    class Graph {
        // số đỉnh = số sách (chỉ số gán khi nạp / thêm sách)
        int n;
        // mảng n danh sách kề (mỗi danh sách là DynamicArray)
        DynamicArray<Edge>* adj;
        DynamicArray<char*> labels;              // isbn của đỉnh i (để in kết
quả)
    public:
        int  addVertex(const char* isbn);
        // vô hướng: thêm vào adj[u] và adj[v]; cộng dồn weight nếu đã có
        void addEdge(int u, int v, int type, int w = 1);
        // đánh dấu xóa (lazy) khi xóa sách
        void removeVertex(int u);
        // trả về các đỉnh cách s ≤ maxDepth theo cạnh type
        DynamicArray<int> bfs(int s, int maxDepth, int type);
        template <typename F> void dfs(int s, F visit, int type);
        const DynamicArray<Edge>& neighbors(int u) const;
        // đếm thành phần liên thông (demo DFS)
        int  components(int type);
    };
```

Xây dựng cạnh: (1) khi thêm sách, nối cạnh loại 0 với mọi sách cùng thể loại – để tránh O(n) mỗi lần, `Library` giữ bảng băm *thể loại → danh sách chỉ số sách*; (2) cạnh loại 1 được dựng lại từ lịch sử giao dịch: với mỗi độc giả, lấy tập sách đã mượn, nối từng cặp và cộng dồn trọng số (`rebuildHistoryEdges`, $O(\Sigma k_i^2)$ với k_i là số sách của độc giả i, nhỏ trong thực tế).

BFS gợi ý cùng thể loại: từ đỉnh sách gốc, duyệt theo cạnh loại 0 với độ sâu ≤ 2 bằng hàng đợi (cài bằng LinkedList), đánh dấu đã thăm bằng mảng bool; kết quả loại sách hết hàng và sắp theo lượt mượn giảm dần. **DFS:** dùng đệ quy để liệt kê thành phần liên thông (nhóm sách cùng thể loại) trong menu demo.

4.8. Cây Trie – autocomplete tên sách (CTDL tự tìm hiểu)

Định nghĩa: Trie (cây tiền tố) là cây có gốc trong đó mỗi cạnh mang một ký tự; đường đi từ gốc đến một nút biểu diễn một chuỗi tiền tố. Nút được đánh dấu `isEnd` nếu tiền tố đó là một khóa đầy đủ. Tìm mọi khóa có tiền tố p: đi xuống |p| bước rồi duyệt cây con.



Hình 4.8. Ví dụ Trie chứa tên sách chuẩn hóa và truy vấn tiền tố "lap"

```
// 'a'..'z' (26) + '0'..'9' (10) + ' ' (1)
const int ALPHABET = 37;
    struct TrieNode {
        TrieNode* children[ALPHABET];                 // nullptr nếu không có
nhánh
        bool isEnd;
```

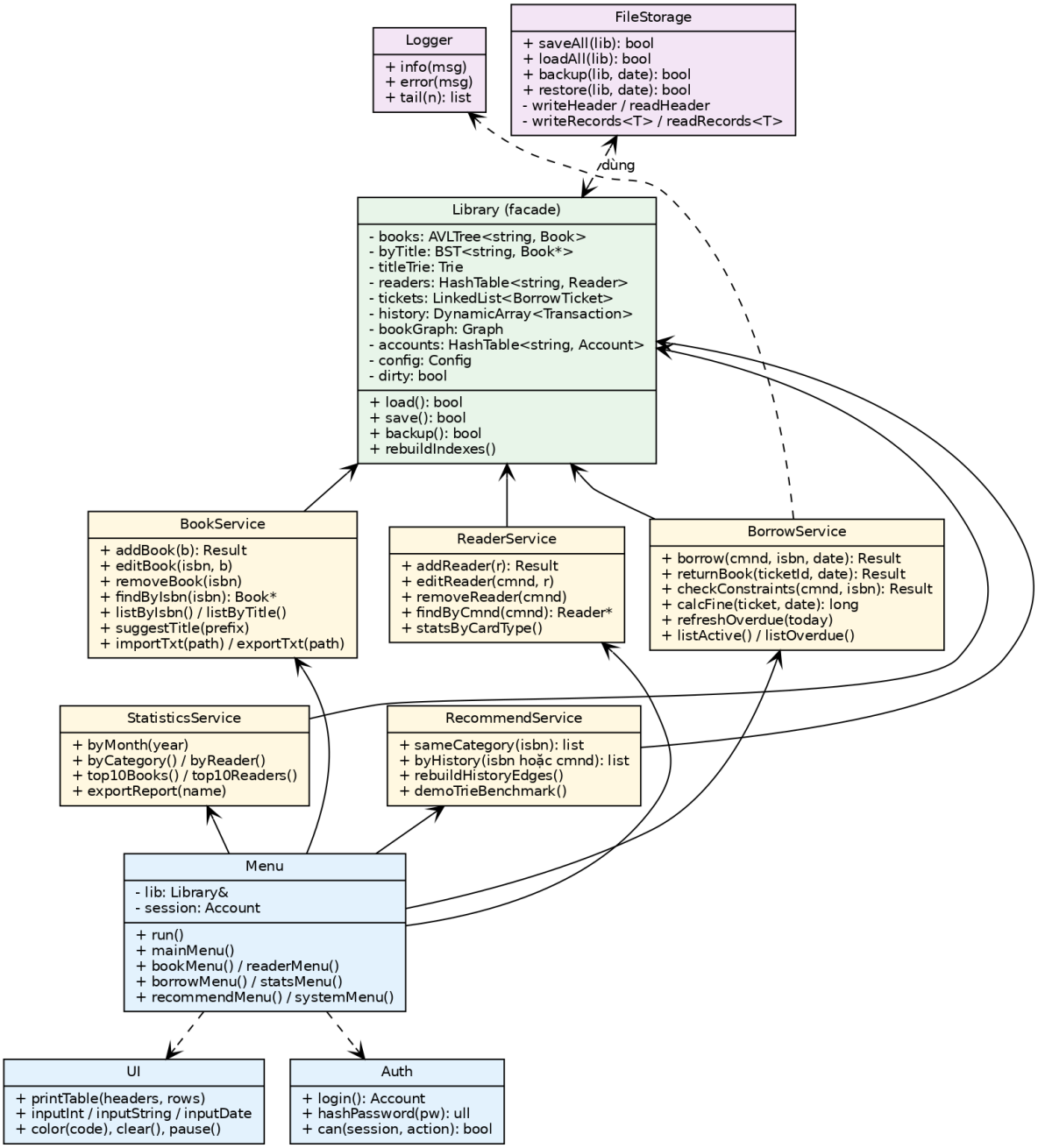
```
        // chỉ số (hoặc ISBN) các sách có đúng tên này
        DynamicArray<int> bookIdx;
    };
    class Trie {
        TrieNode* root; int nodeCount;
        // a-z -> 0..25, 0-9 -> 26..35, ' ' -> 36, khác -> -1 (bỏ qua)
        static int charIndex(char c);
        // DFS thu kết quả
        void collect(TrieNode* n, std::string& cur, DynamicArray<std::string>&
out, int limit);
        // cắt nhánh rỗng
        bool removeRec(TrieNode* n, const std::string& w, int depth, int bookIdx);
    public:
        void insert(const std::string& word, int bookIdx);            // O(L)
        bool remove(const std::string& word, int bookIdx);            // O(L)
        // O(|p| + k·L)
        DynamicArray<std::string> startsWith(const std::string& prefix, int limit
= 20);
        bool exists(const std::string& word) const;
        int  nodes() const; size_t memoryBytes() const;               // phục vụ
so sánh
    };
```

Xử lý tiếng Việt: tên sách được chuẩn hóa bằng `Utils::normalize` (bỏ dấu: “Lập trình C++” → “lap trinh c++” → “lap trinh c”) trước khi chèn và trước khi tìm, nên người dùng gõ có dấu hay không dấu đều ra kết quả; bảng chữ cái 37 ký hiệu giữ mỗi nút nhỏ (37 con trỏ ≈ 300 byte).

So sánh (thực hiện tuần 8–9): cùng tập N tên sách và 1.000 truy vấn tiền tố: Trie O(|p| + k) so với BST `prefixSearch` (đi đến nút đầu tiên ≥ p rồi duyệt in-order cho đến khi hết tiền tố, O(log n + k) nhưng hằng số so sánh chuỗi lớn) và tìm tuyến tính O(N·L). Chỉ tiêu: thời gian, bộ nhớ (`memoryBytes`), số phép so sánh ký tự.

CHƯƠNG 5. THIẾT KẾ LỚP LIBRARY, SERVICE VÀ GIAO DIỆN

5.1. Sơ đồ lớp ứng dụng



Hình 5.1. Sơ đồ lớp tầng ứng dụng: Library (facade), các Service, Menu và lưu trữ

5.2. Lớp Library (facade)

```
class Library {
    public:
        AVLTree<std::string, Book>          books;        // nguồn dữ liệu sách
        BST<std::string, Book*>             byTitle;      // chỉ mục theo tên
chuẩn hóa
        Trie                                titleTrie;    // autocomplete
        HashTable<std::string, Reader>      readers;
```

```
        // thể loại -> chỉ số đỉnh (để nối cạnh nhanh)
        HashTable<std::string, DynamicArray<int>> byCategory;
        LinkedList<BorrowTicket>            tickets;
        DynamicArray<Transaction>           history;
        Graph                               bookGraph;
        HashTable<std::string, Account>     accounts;
        Config                              config;
        long nextTicketNo, nextTransId; bool dirty;
        bool load(); bool save(); bool backup(); bool restore(const std::string&
date);
        // sau khi load: dựng byTitle, titleTrie, byCategory, bookGraph từ books +
history
        void rebuildIndexes();
        void markDirty() { dirty = true; }
    };
```

`Library` chỉ giữ dữ liệu và việc nạp / lưu; mọi logic nghiệp vụ nằm trong các Service nhận `Library&` qua constructor. Cách này giúp unit test từng Service với một `Library` giả lập trong bộ nhớ mà không chạm file.

5.3. Các lớp Service

Bảng 5.1. Các lớp Service và trách nhiệm

Lớp	Phương thức chính	CTDL sử dụng	Yêu cầu
BookService	addBook, editBook, removeBook, findByIsbn, listByIsbn, listByTitle, suggestTitle, importTxt, exportTxt/Csv	AVL, BST, Trie, Graph, byCategory	FR-B-01..09
ReaderService	addReader, editReader, renewCard, lockReader, removeReader, findByCmnd, listAll, statsByCardType	HashTable	FR-R-01..06
BorrowService	checkConstraints, borrow, returnBook, calcFine, refreshOverdue, listActive, listOverdue, historyOf	LinkedList, DynArray, AVL, HashTable, Graph	FR-L-01..06
StatisticsService	byMonth, byCategory, byReader, top10Books, top10Readers, fineByMonth, exportReport	DynArray, AVL, HashTable (bảng đếm tạm)	FR-S-01..07
RecommendService	sameCategory, byHistoryOfBook, byHistoryOfReader, rebuildHistoryEdges, demoTrieBenchmark, dfsComponents	Graph, DynArray, Trie, BST	FR-G-01..05

5.4. Lớp Menu, UI, Auth

- `Menu`: vòng lặp chính `run()`, các hàm `bookMenu()`, `readerMenu()`, `borrowMenu()`, `statsMenu()`, `recommendMenu()`, `systemMenu()`; mỗi mục menu là một hàm riêng (≤ 40 dòng) chỉ làm 3 việc: nhập – gọi service – hiển thị `Result`.

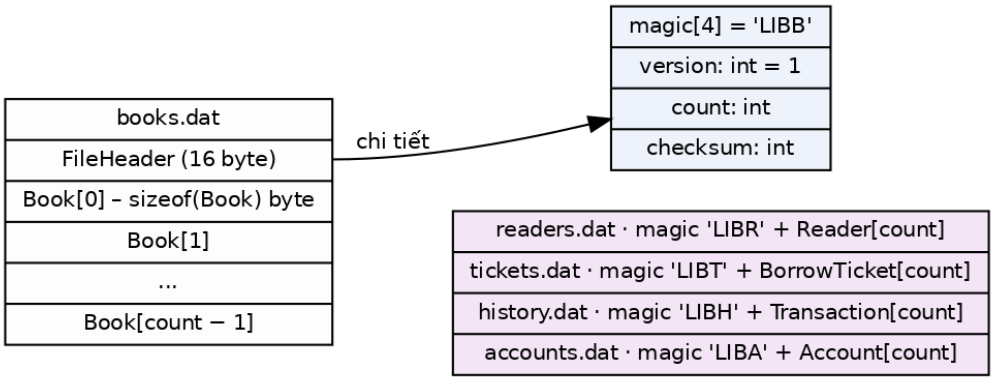
- `UI`: `printTable(headers, rows, widths)` căn cột theo độ rộng ký tự hiển thị (UTF-8); `inputInt(min, max)`, `inputString(maxLen, pattern)`, `inputDate()` lặp tối đa 3 lần; `color()` dùng mã ANSI, tắt được qua `config.useColor`; `paginate(rows, 20)`.
- `Auth`: `login()` hỏi tài khoản / mật khẩu (ẩn ký tự), băm FNV-1a với salt cố định, so khớp trong `accounts`; `can(session, action)` tra bảng quyền tĩnh theo vai trò; lần chạy đầu tạo tài khoản `admin/admin` và buộc đổi mật khẩu.

Bảng 5.2. Ma trận phân quyền theo vai trò

Hành động	Admin	Thủ thư	Độc giả
Thêm / sửa / xóa sách, độc giả			–
Lập phiếu mượn, trả sách			–
Thống kê, xuất báo cáo			–
Tìm sách, autocomplete, gợi ý			
Xem phiếu mượn của mình			(chỉ của mình)
Import / export danh mục, sao lưu / phục hồi		(chỉ sao lưu)	–
Quản lý tài khoản, xem log		–	–

CHƯƠNG 6. THIẾT KẾ LƯU TRỮ FILE

6.1. Định dạng file nhị phân



Hình 6.1. Bố cục file nhị phân: header + mảng bản ghi kích thước cố định

```
struct FileHeader { char magic[4]; int version; int count; int checksum; };   //
16 byte

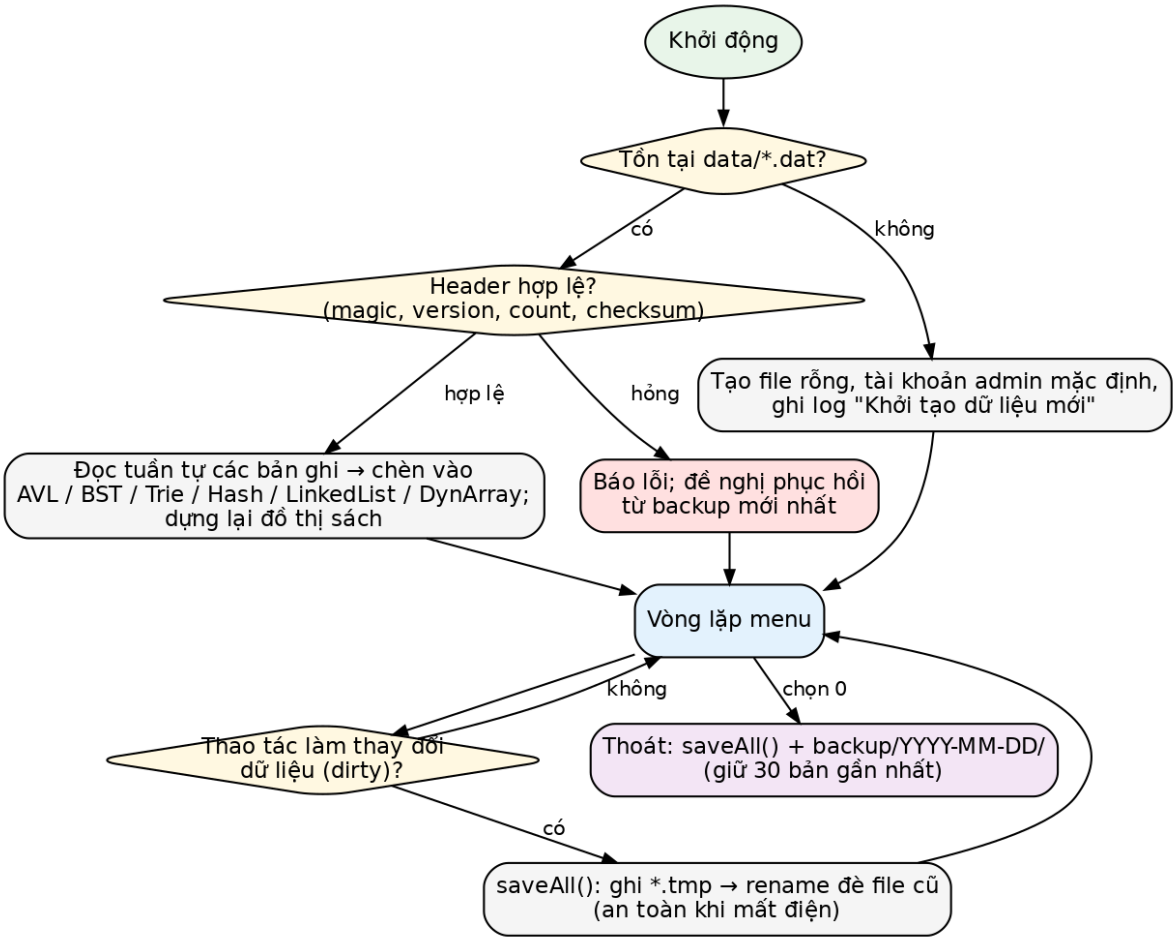
    template <typename T>
    bool FileStorage::writeRecords(const char* path, const char* magic, const T*
data, int count) {
        std::string tmp = std::string(path) + ".tmp";
        FILE* f = fopen(tmp.c_str(), "wb");  if (!f) return false;
        FileHeader h = {{magic[0],magic[1],magic[2],magic[3]}, FORMAT_VERSION,
count, checksumOf(data, count)};
        bool ok = fwrite(&h, sizeof h, 1, f) == 1 && (count == 0 || fwrite(data,
sizeof(T), count, f) == (size_t)count);
        fclose(f);
        if (!ok) { remove(tmp.c_str()); return false; }
        // đổi tên nguyên tử: file cũ còn nguyên nếu lỗi
        remove(path); return rename(tmp.c_str(), path) == 0;
    }
```

Bảng 6.1. Các file nhị phân

File	magic	Kiểu bản ghi	Nguồn trong Library	Ghi chú
books.dat	LIBB	Book	books (AVL, duyệt in-order → mảng tạm)	Ghi theo thứ tự ISBN; khi nạp chèn lại vào AVL (thứ tự tăng dần là trường hợp AVL phải quay nhiều – kiểm tra tính đúng)
readers.dat	LIBR	Reader	readers (forEach → mảng tạm)	
tickets.dat	LIBT	BorrowTicket	tickets (forEach → mảng tạm)	Thứ tự thêm = thứ tự mã phiếu
history.dat	LIBH	Transaction	history.raw()	Ghi trực tiếp cả khối
accounts.dat	LIBA	Account	accounts	

Khi đọc: kiểm tra `magic`, `version == FORMAT_VERSION`, kích thước file = 16 + count × sizeof(T) và checksum (tổng byte mod 2^{31}). Sai bất kỳ điều kiện nào → trả lỗi "file hỏng", chương trình đề nghị phục hồi từ backup. Thứ tự nạp: books → readers → accounts → tickets → history, sau đó `rebuildIndexes()`.

6.2. Luồng nạp / lưu / sao lưu

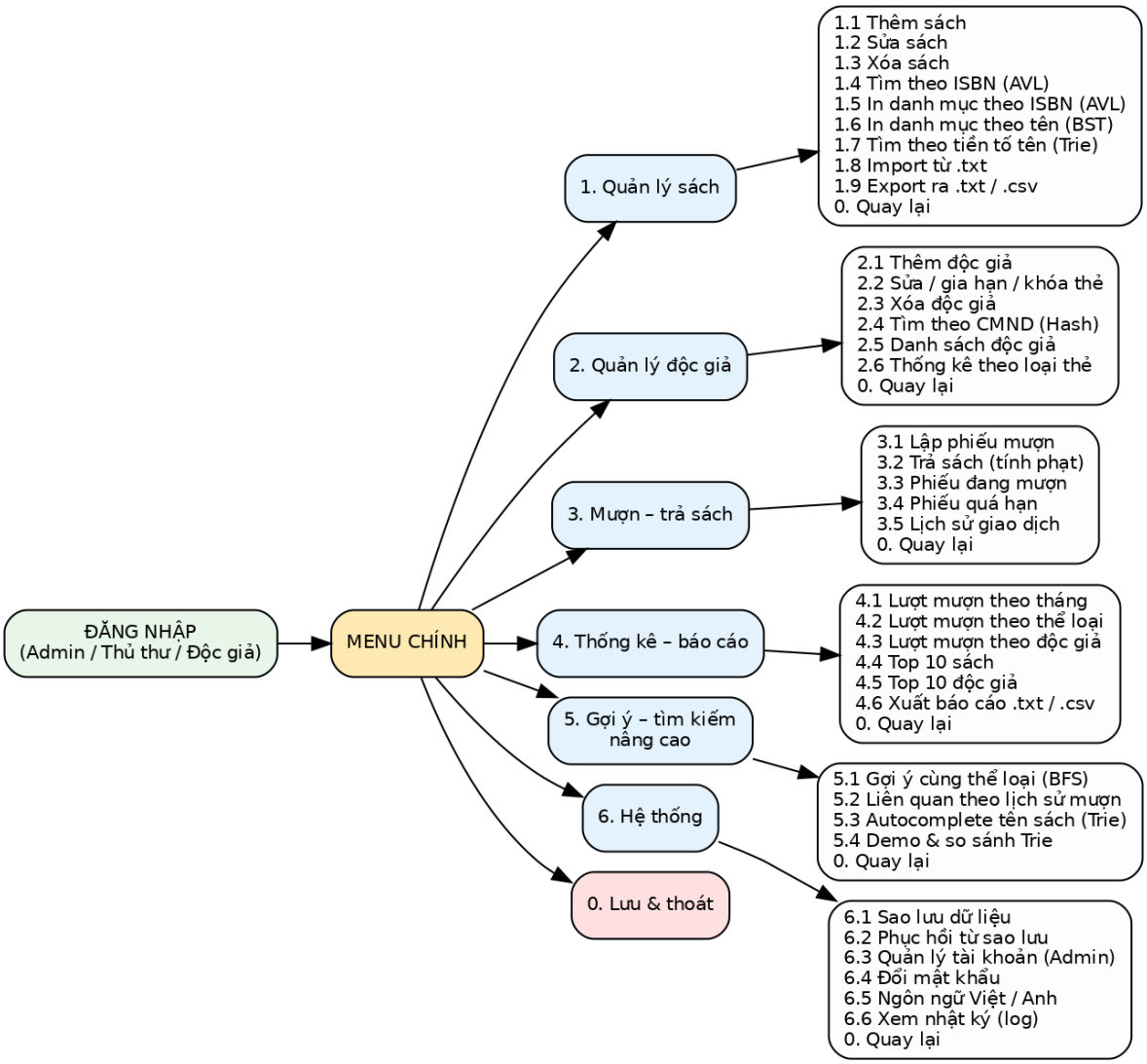


Hình 6.2. Luồng nạp dữ liệu khi khởi động, lưu khi thay đổi và sao lưu khi thoát

- **Khi nào lưu:** Service đặt `lib.dirty = true` sau thao tác ghi; `Menu` gọi `lib.save()` ngay sau khi Service trả về `ok` (FR-F-02) và khi thoát.
- **Sao lưu theo ngày:** `backup()` tạo thư mục `data/backup/YYYY-MM-DD/` (nếu chưa có) và sao chép 5 file .dat; giữ lại 30 thư mục mới nhất (xóa cũ hơn theo tên thư mục sắp xếp).
- **Phục hồi:** liệt kê thư mục backup, người dùng chọn, xác nhận; sao chép đè rồi `load()` lại; ghi log.
- **Nhật ký:** `Logger` mở `data/log.txt` ở chế độ append, mỗi dòng: `[2026-10-19 14:05:33] [thuthu01] BORROW PM00031 cmnd=079... isbn=978... OK`.
- **Import / export văn bản:** dòng `ISBN|Tên|Tác giả|Thể loại|Năm|SL`; bỏ qua dòng bắt đầu bằng #; báo số dòng lỗi và lý do.

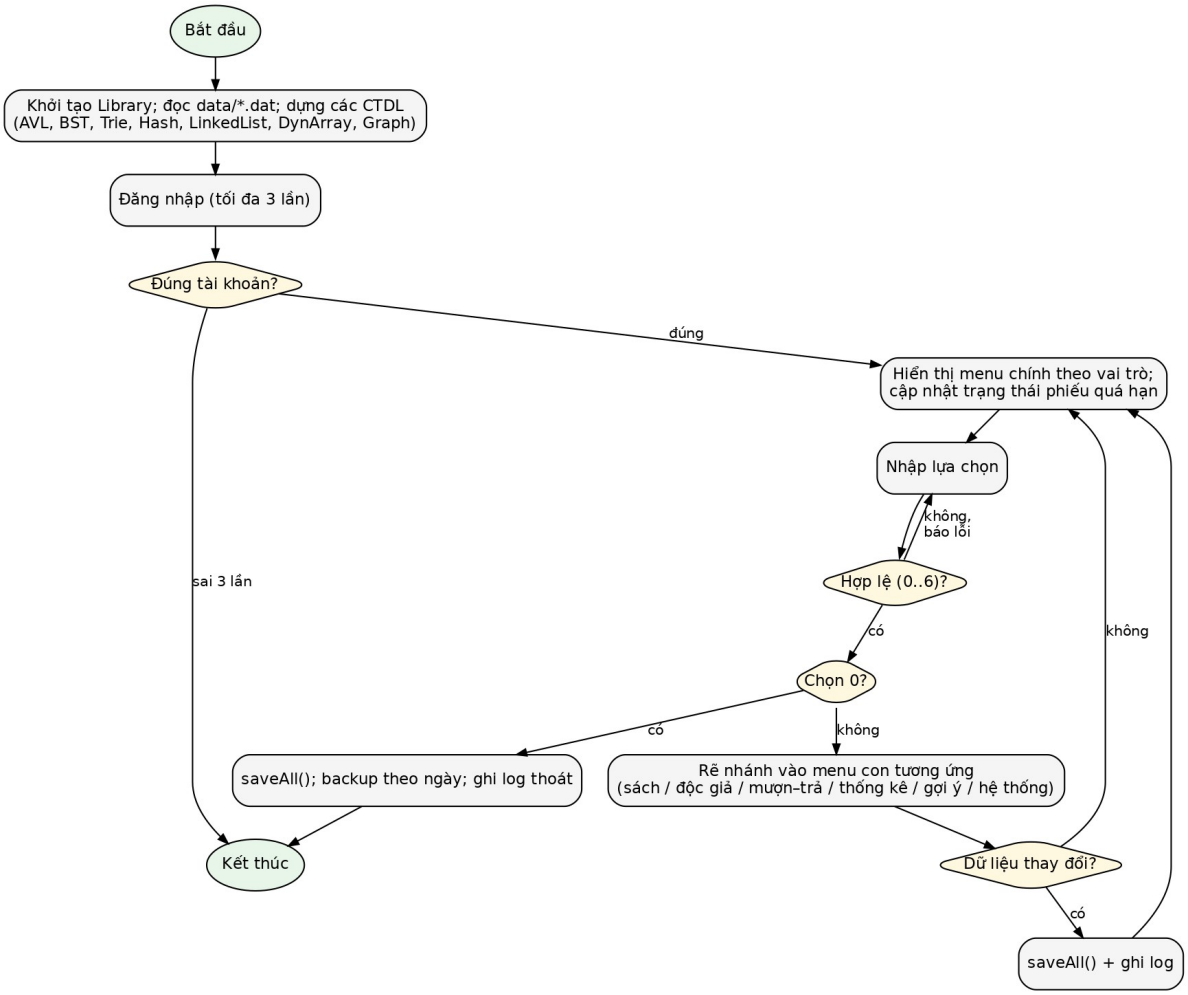
CHƯƠNG 7. THIẾT KẾ MENU VÀ LUỒNG CHƯƠNG TRÌNH

7.1. Cây menu



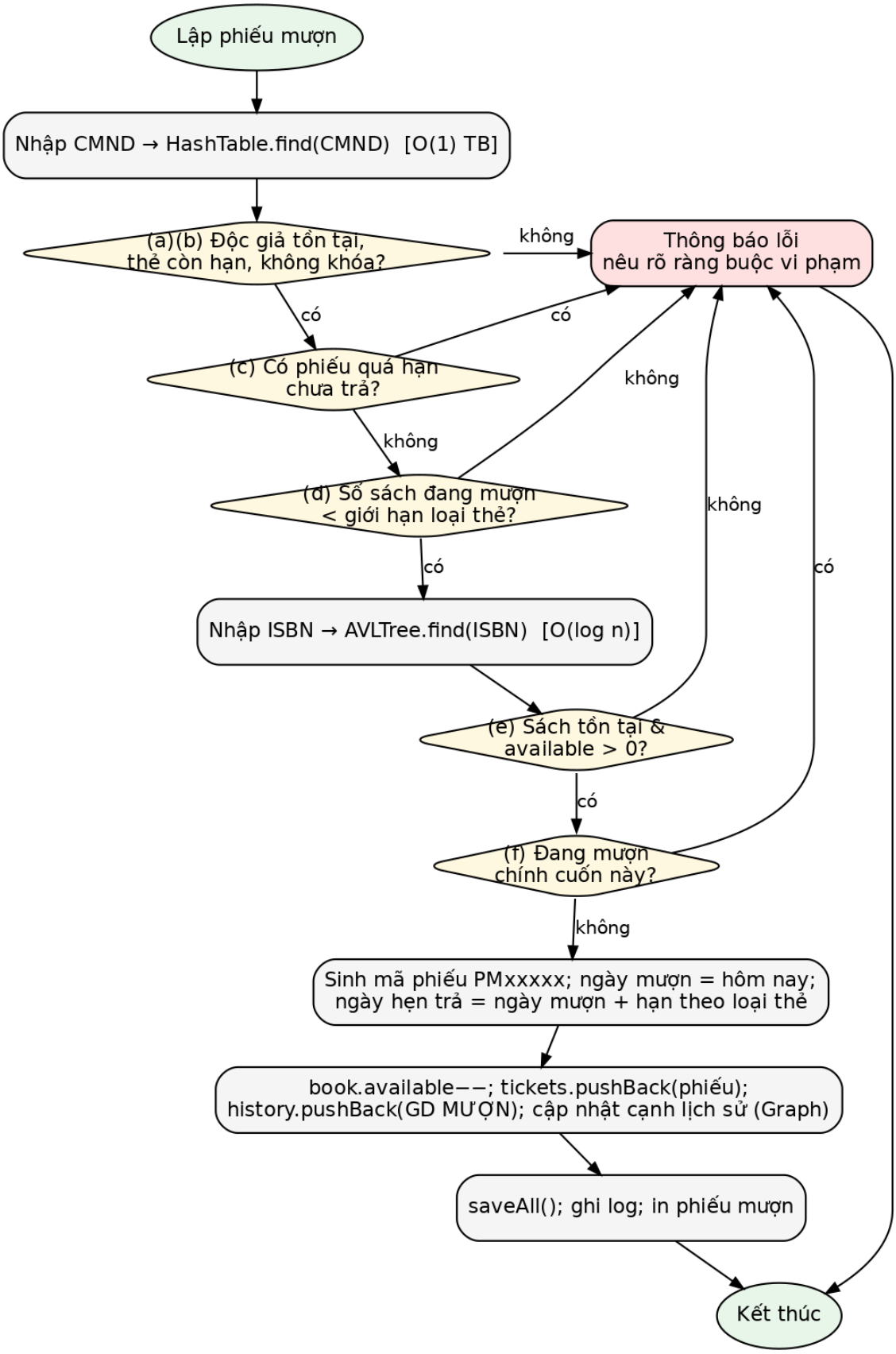
Hình 7.1. Cây menu chính và các menu con

7.2. Luồng chính của chương trình



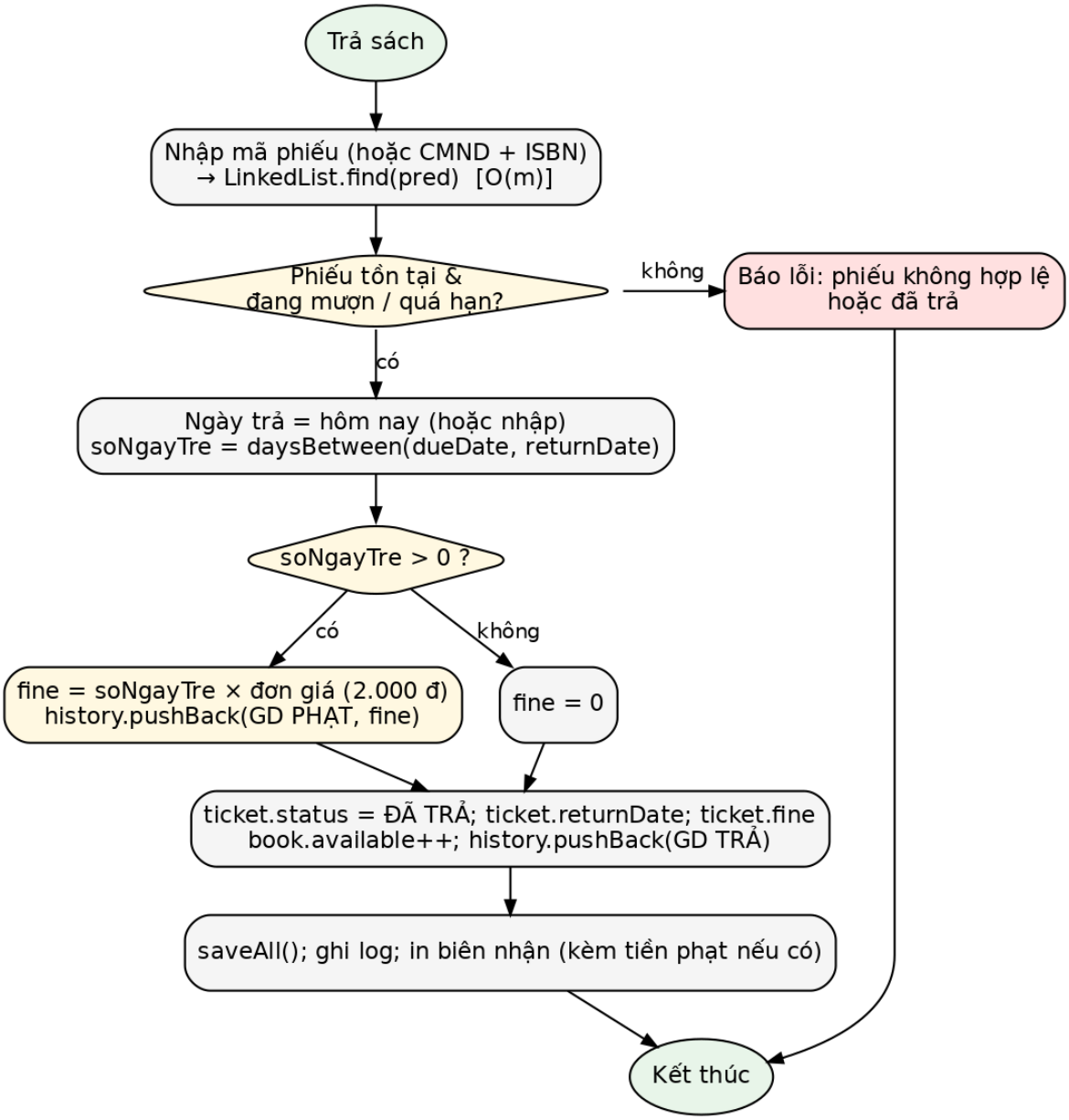
Hình 7.2. Lưu đồ luồng chính: khởi động → đăng nhập → vòng lặp menu → lưu & thoát

7.3. Luồng lập phiếu mượn



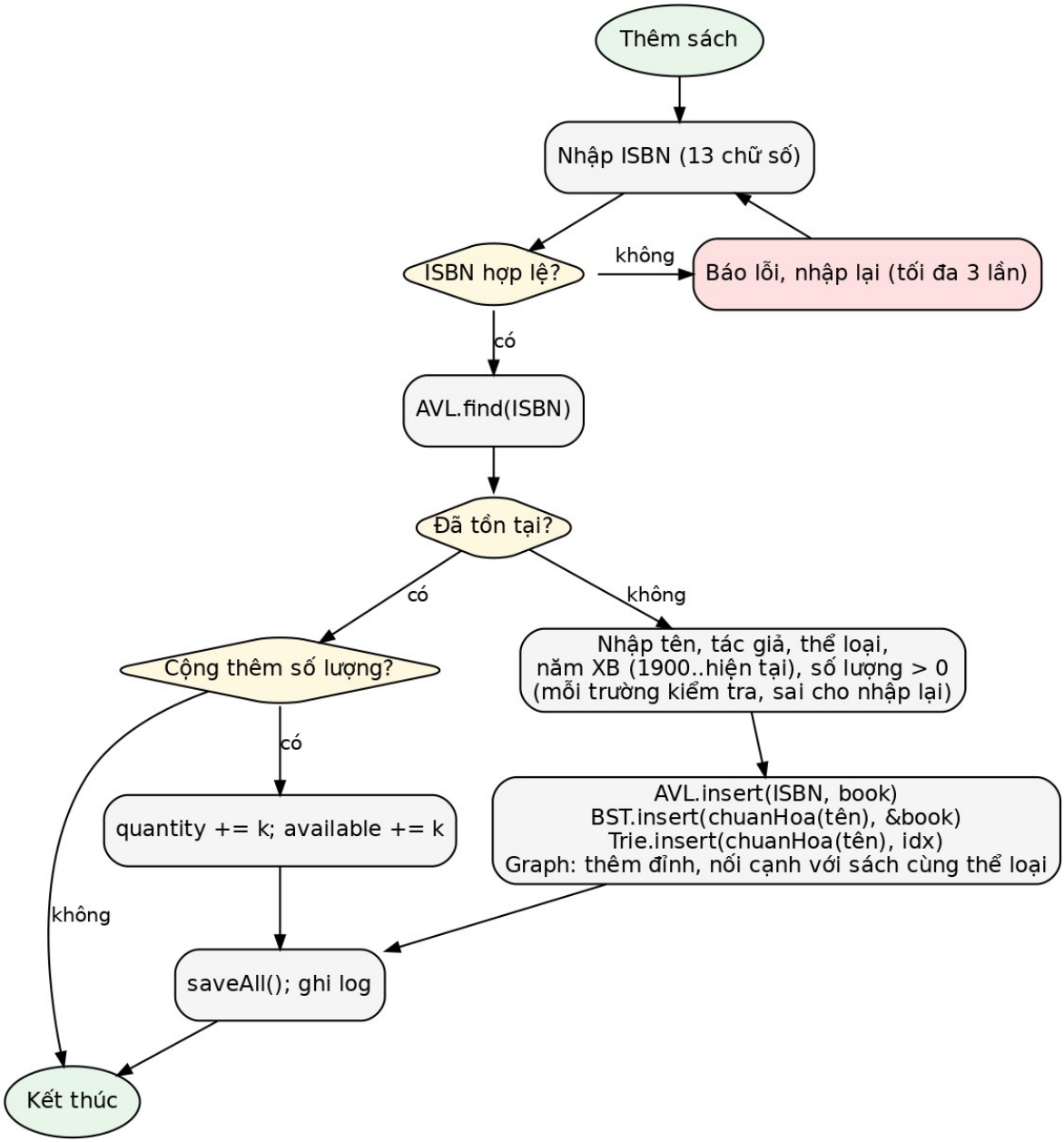
Hình 7.3. Lưu đồ lập phiếu mượn với 6 ràng buộc (a)–(f) theo FR-L-03

7.4. Luồng trả sách và tính phạt



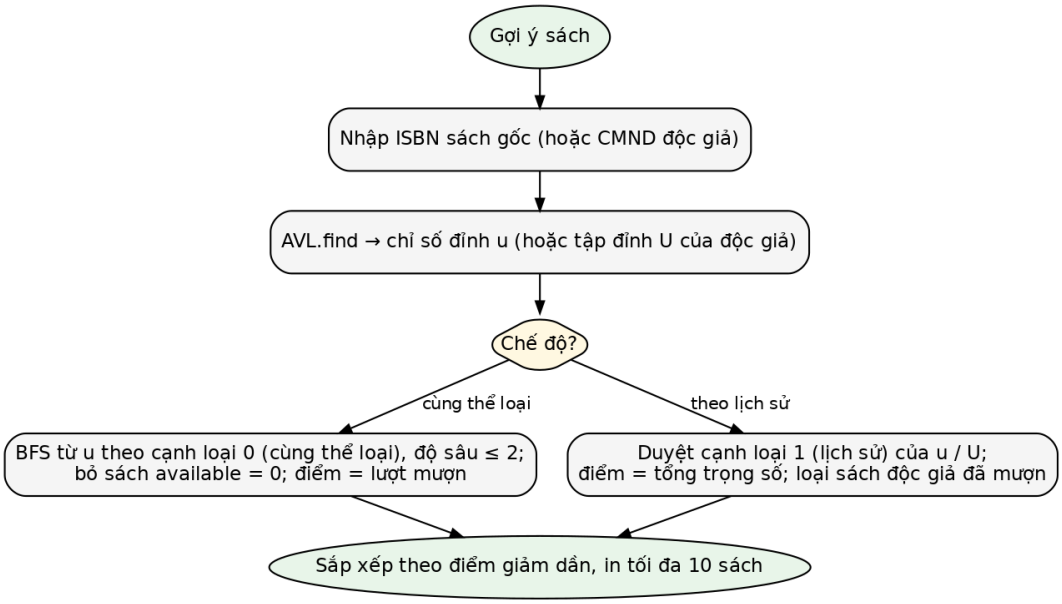
Hình 7.4. Lưu đồ trả sách và tính tiền phạt quá hạn

7.5. Luồng thêm sách



Hình 7.5. Lưu đồ thêm sách: cập nhật đồng thời AVL, BST, Trie và đồ thị

7.6. Luồng gợi ý sách



Hình 7.6. Lưu đồ gợi ý sách cùng thể loại (BFS) và theo lịch sử mượn

7.7. Phác thảo màn hình

```
┌──────────────────────────── QUẢN LÝ SÁCH ────────────────────────────┐
    │ 1. Thêm sách      4. Tìm theo ISBN        7. Tìm theo tiền tố tên    │
    │ 2. Sửa sách       5. In danh mục (ISBN)   8. Import từ file          │
    │ 3. Xóa sách       6. In danh mục (tên)    9. Export ra file          │
    │ 0. Quay lại                                                          │
    └──────────────────────────────────────────────────────────────────────┘

    Chọn [0-9]: 7
    Nhập tiền tố tên sách: lap
    Có 3 kết quả (0.02 ms, Trie):
      # | ISBN          | Tên sách                       | Tác giả        |
Còn/Tổng
      1 | 9786041234567 | Lập trình C++ căn bản          | Nguyễn Văn A   |  3/5
      2 | 9786049876543 | Lập trình hướng đối tượng      | Trần Thị B     |  0/2
      3 | 9786045556667 | Lập trình Python cho người mới | Lê Văn C       |  4/4
    Chọn số thứ tự để xem chi tiết (0 = bỏ qua): _

┌──────────────────────────── PHIẾU MƯỢN ──────────────────────────────┐
    │ Mã phiếu : PM00031              Ngày mượn : 19/10/2026               │
    │ Độc giả  : 079012345678 – Nguyễn Văn A (Sinh viên)                   │
    │ Sách     : 9786041234567 – Lập trình C++ căn bản                     │
    │ Hẹn trả  : 02/11/2026 (14 ngày)   Đang mượn: 2/3 cuốn                │
    └──────────────────────────────────────────────────────────────────────┘

       Lập phiếu thành công. Dữ liệu đã được lưu.
```

CHƯƠNG 8. THIẾT KẾ GIẢI THUẬT CHÍNH (MÃ GIẢ)

8.1. Kiểm tra ràng buộc và lập phiếu mượn

```
Result BorrowService::checkConstraints(cmnd, isbn, today):
        r = lib.readers.find(cmnd)
        // (a)
        if r == null                      -> fail("Không tìm thấy độc giả")
        // (b)
        if r.status == LOCKED             -> fail("Thẻ đang bị khóa")
        // (b)
        if today > r.expDate              -> fail("Thẻ hết hạn, cần gia hạn")
        overdue = lib.tickets.countIf(t -> t.cmnd == cmnd && t.status == OVERDUE)
        // (c)
        if overdue > 0                    -> fail("Đang có " + overdue + " phiếu
quá hạn")
        borrowing = lib.tickets.countIf(t -> t.cmnd == cmnd && t.status !=
RETURNED)
        // (d)
        if borrowing >= config.maxBooks[r.cardType] -> fail("Đã mượn tối đa ...")
        b = lib.books.find(isbn)
        // (e)
        if b == null                      -> fail("Không tìm thấy sách")
        // (e)
        if b.available <= 0               -> fail("Sách đã được mượn hết")
        if lib.tickets.find(t -> t.cmnd == cmnd && t.isbn == isbn && t.status !=
RETURNED)
                                          // (f)
                                          -> fail("Độc giả đang mượn chính cuốn
này")
        return success()

    Result BorrowService::borrow(cmnd, isbn, date):
        chk = checkConstraints(cmnd, isbn, date); if !chk.ok return chk
        t.id = "PM" + pad5(lib.nextTicketNo++); t.cmnd, t.isbn
        t.borrowDate = date; t.dueDate = date.addDays(config.loanDays[r.cardType])
        t.status = BORROWING; t.fine = 0
        b.available--                                   // sửa trực tiếp bản ghi
trong AVL
        lib.tickets.pushBack(t)
        lib.history.pushBack(Transaction{BORROW, t.id, cmnd, isbn, date, 0})
        lib.bookGraph: với mỗi sách s' độc giả này từng mượn: addEdge(idx(isbn),
idx(s'), type=1, +1)
        lib.markDirty(); logger.info(...)
        return success("Lập phiếu " + t.id)
```

8.2. Trả sách và tính phạt

```
long BorrowService::calcFine(ticket, returnDate):
        late = Date::daysBetween(ticket.dueDate, returnDate)
        return late > 0 ? late * config.finePerDay : 0

    Result BorrowService::returnBook(ticketId, returnDate):
        t = lib.tickets.find(x -> x.id == ticketId)
        if t == null || t.status == RETURNED -> fail("Phiếu không hợp lệ hoặc đã
trả")
        t.fine = calcFine(*t, returnDate); t.returnDate = returnDate; t.status =
RETURNED
        lib.books.find(t.isbn).available++
        lib.history.pushBack({RETURN, ...}); if t.fine > 0:
lib.history.pushBack({FINE, ..., amount = t.fine})
        lib.markDirty(); return success(t.fine > 0 ? "Trả trễ, phạt " + t.fine + "
đ" : "Trả đúng hạn")

    // gọi khi khởi động và mỗi lần vào menu mượn–trả
    void BorrowService::refreshOverdue(today):
```

```
        lib.tickets.forEach(t -> if t.status == BORROWING && today > t.dueDate:
t.status = OVERDUE)
```

8.3. Top 10 sách / độc giả

```
DynamicArray<Pair> StatisticsService::top10Books(from, to):
        HashTable<string,int> cnt(2003)
        // O(T)
        for each tr in lib.history: if tr.type == BORROW && from <= tr.date <= to:
cnt[tr.isbn]++
        // DynamicArray<Pair{isbn, count}>
        arr = cnt.toArray()
        // merge sort O(k log k), k = số sách có lượt mượn
        arr.sort((a,b) -> a.count > b.count)
        return arr.first(10)                                  // kèm tên sách tra từ
AVL
```

8.4. Gợi ý sách cùng thể loại (BFS)

```
DynamicArray<int> Graph::bfs(s, maxDepth, type):
        visited[0..n) = false; dist[s] = 0; queue.push(s); visited[s] = true
        while queue not empty:
            u = queue.pop()
            if dist[u] == maxDepth: continue
            for e in adj[u]: if e.type == type && !visited[e.to]:
                visited[e.to] = true; dist[e.to] = dist[u] + 1; queue.push(e.to);
result.push(e.to)
        return result                                         // O(V + E)

    RecommendService::sameCategory(isbn):
        u = idx(isbn); cand = lib.bookGraph.bfs(u, 2, 0)
        lọc cand: available > 0; sắp theo lượt mượn giảm dần (đếm từ history); lấy
10
```

8.5. Autocomplete bằng Trie

```
DynamicArray<string> Trie::startsWith(prefix, limit):
        p = normalize(prefix); node = root
        for c in p: i = charIndex(c); if i < 0 continue; if node.children[i] ==
null return {}; node = node.children[i]
        // DFS cây con, dừng khi đủ limit
        collect(node, p, out, limit); return out

    Trie::collect(node, cur, out, limit):
        if out.size() >= limit return
        if node.isEnd: out.push(cur)
        for i in 0..36: if node.children[i]: collect(node.children[i], cur +
charOf(i), out, limit)
```

8.6. Chuẩn hóa chuỗi tiếng Việt

```
string Utils::normalize(s):
      // 1. chuyển UTF-8 về mã điểm (code point); 2. ánh xạ bảng 134 ký tự có dấu
-> không dấu (à á ạ ả ã â ... -> a, đ -> d)

        // 3. chữ thường; 4. giữ a-z, 0-9, khoảng trắng; gộp nhiều khoảng trắng;
cắt hai đầu
        // Ví dụ: "Lập Trình  C++ Căn Bản" -> "lap trinh c can ban"
```

CHƯƠNG 9. THIẾT KẾ XỬ LÝ LỖI VÀ KIỂM THỬ

9.1. Xử lý lỗi và kiểm tra dữ liệu nhập

Bảng 9.1. Thiết kế xử lý lỗi

Tình huống	Phát hiện ở	Xử lý
Nhập ISBN / CMND / SĐT sai định dạng, tên rỗng, năm ngoài khoảng, số lượng ≤ 0	UI::input*	Báo lỗi cụ thể, cho nhập lại tối đa 3 lần, sau đó hủy thao tác về menu
Chuỗi nhập dài hơn giới hạn	UI::inputString	Cắt theo giới hạn và cảnh báo; luôn dùng `strncpy` + kết thúc '\0'
Vi phạm quy tắc nghiệp vụ (trùng khóa, xóa sách đang mượn, ràng buộc mượn)	Service	Trả `Result::fail(message)`; Menu in màu đỏ; không thay đổi dữ liệu
File dữ liệu thiếu	FileStorage::loadAll	Tạo file rỗng, ghi log, tiếp tục chạy
File hỏng (magic / version / kích thước / checksum sai)	FileStorage::readRecords	Không nạp file đó; thông báo và đề nghị phục hồi backup; cho phép tiếp tục với dữ liệu rỗng
Ghi file thất bại (hết đĩa, không có quyền)	FileStorage::writeRecords	Giữ file cũ (nhờ ghi .tmp + rename), báo lỗi, giữ `dirty = true` để thử lại khi thoát
Cấp phát bộ nhớ thất bại	new	Bắt `std::bad_alloc` ở `main`, lưu khẩn cấp rồi thoát có kiểm soát
Nhập ký tự khi chờ số, EOF (Ctrl+Z / Ctrl+D)	UI::inputInt	Xóa trạng thái lỗi của `cin`, bỏ dòng, nhập lại; EOF → coi như chọn thoát và lưu

9.2. Kế hoạch kiểm thử

Bảng 9.2. Kế hoạch kiểm thử

Loại	File test	Nội dung kiểm tra	Tuần
Unit	test_date.cpp	isValid, addDays qua năm nhuận, daysBetween	3
Unit	test_avl.cpp	Chèn 10.000 khóa tăng dần → chiều cao ≤ $1{,}44 \cdot \log_2 n$; xóa ngẫu nhiên giữ cân bằng; find đúng / sai	3
Unit	test_bst.cpp	Chèn / xóa 3 trường hợp; in-order tăng dần; khóa trùng	3
Unit	test_hash.cpp	Chèn 5.000 CMND, không mất phần tử; va chạm; rehash giữ nguyên dữ liệu	4
Unit	test_linkedlist.cpp, test_dynarray.cpp	pushBack, find, remove đầu / giữa / cuối; grow; sort	5

Loại	File test	Nội dung kiểm tra	Tuần
Unit	test_storage.cpp	Ghi rồi đọc lại bằng nhau; file hỏng bị phát hiện; .tmp + rename	5
Integration	test_borrow.cpp	6 ràng buộc (a)–(f); phạt 0 / 1 / 5 ngày trễ; available tăng giảm đúng	6
Unit	test_graph.cpp	BFS độ sâu 1, 2; DFS thành phần liên thông; trọng số cộng dồn	7
Unit	test_trie.cpp	Chèn / xóa; startsWith có / không có kết quả; tiếng Việt có dấu; limit	8
Hiệu năng	benchmark.cpp	AVL vs BST, Hash vs tuần tự, Trie vs BST vs tuyến tính; N = 10^3, 10^4, 10^5; xuất CSV để vẽ biểu đồ	9
Thủ công	docs/test_cases.md	Kịch bản theo từng mục menu, ảnh chụp màn hình cho báo cáo chương 4	9

Mỗi file test là một chương trình độc lập dùng macro `ASSERT(cond, msg)` tự viết (không cần framework), trả về mã thoát khác 0 khi thất bại để `make test` dừng.

CHƯƠNG 10. KẾ HOẠCH CÀI ĐẶT THEO THIẾT KẾ

Bảng 10.1. Kế hoạch cài đặt theo tuần

Tuần	Module cài đặt	File	Tiêu chí hoàn thành
3	Date, Book, Utils::normalize, AVLTree, BST, BookService (thêm / xóa / sửa / tìm / in), Menu sách	Date.*, Book.h, Utils.*, AVLTree.h, BST.h, BookService.*, MenuBook.cpp	Quản lý sách chạy trên bộ nhớ; test_avl, test_bst xanh
4	Reader, HashTable, ReaderService, Menu độc giả, benchmark Hash vs tuần tự	Reader.h, HashTable.h, ReaderService.*, MenuReader.cpp	Quản lý độc giả; test_hash xanh; bảng so sánh sơ bộ
5	BorrowTicket, Transaction, LinkedList, DynamicArray, FileStorage, Logger, Library::load/save/backup	LinkedList.h, DynamicArray.h, FileStorage.*, Logger.*, Library.*	Thoát và chạy lại dữ liệu còn nguyên; mốc #2
6	BorrowService (ràng buộc, mượn, trả, phạt, quá hạn), Menu mượn – trả, Config	BorrowService.*, MenuBorrow.cpp, Config.*	test_borrow xanh; mốc #3 phần mượn – trả
7	Graph (BFS/DFS), RecommendService, StatisticsService, Exporter, Menu thống kê / gợi ý	Graph.*, RecommendService.*, StatisticsService.*, Exporter.*, MenuStats.cpp	Gợi ý + top 10 + xuất báo cáo; mốc #3
8	Trie, tích hợp autocomplete, menu demo Trie, Auth / phân quyền, màu giao diện	Trie.*, Auth.*, UI.*, MenuSystem.cpp	Demo Trie; đăng nhập 3 vai trò
9	Unit test đầy đủ, benchmark, sửa lỗi, dữ liệu mẫu ≥ 50/20/30, user manual, CSV, đa ngôn ngữ	tests/*, data/sample/*, docs/manual.md	make test xanh; bảng số liệu + biểu đồ; mốc #4

10.1. Các điểm cần giảng viên góp ý tại mốc #1

1. Lựa chọn Trie làm CTDL tự tìm hiểu và cách chuẩn hóa tiếng Việt (bỏ dấu) với bảng chữ cái 37 ký hiệu.
2. Cách lưu phiếu đã trả trong cùng danh sách liên kết với phiếu đang mượn (thay vì tách hai danh sách).
3. Hai loại cạnh trong một đồ thị (cùng thể loại / cùng được mượn) thay vì hai đồ thị riêng.
4. Mức độ cần thiết của tài khoản – phân quyền (mở rộng) so với thời gian còn lại.
5. Đơn giá phạt 2.000 đ/ngày và giới hạn mượn theo loại thẻ (có thể cấu hình).

PHỤ LỤC

PL.1. Ma trận truy vết thiết kế – yêu cầu

Bảng PL.1. Ma trận truy vết thiết kế – yêu cầu

Yêu cầu (SRS)	Thành phần thiết kế	Mục
FR-B-01..09	AVLTree, BST, Trie, BookService, MenuBook, Exporter	4.2, 4.3, 4.8, 5.3, 7.5
FR-R-01..06	HashTable, ReaderService, MenuReader	4.4, 5.3
FR-L-01..06	LinkedList, DynamicArray, BorrowService, lưu đồ mượn / trả, mã giả	4.5, 4.6, 7.3, 7.4, 8.1, 8.2
FR-S-01..07	DynamicArray, StatisticsService, Exporter, mã giả top 10	4.6, 5.3, 8.3
FR-G-01..05	Graph, Trie, RecommendService, lưu đồ gợi ý, mã giả BFS / Trie	4.7, 4.8, 7.6, 8.4, 8.5
FR-F-01..05	FileStorage, Logger, định dạng file, luồng nạp / lưu	6.1, 6.2
FR-X-01..05	Auth, ma trận phân quyền, UI màu, CSV, tests/	5.4, 9.2
BR-01..10	Config, checkConstraints, calcFine	3.4, 8.1, 8.2
NFR-01..09	Độ phức tạp từng CTDL, ghi .tmp + rename, xử lý lỗi, Makefile	4.1, 6.1, 9.1

PL.2. Quy ước đặt tên và mã nguồn

- Lớp / struct: `PascalCase`; hàm và biến: `camelCase`; hằng số và enum: `UPPER_SNAKE_CASE`; file trùng tên lớp.
- Mỗi hàm public có comment mô tả ngắn, tham số và giá trị trả về; hàm phức tạp (xoay AVL, xóa nút, rehash, BFS) có comment giải thích từng bước.
- Biên dịch với `-Wall -Wextra` không cảnh báo; không rò rỉ bộ nhớ (kiểm tra bằng valgrind trên Linux ở tuần 9).
- Commit theo quy ước: `feat: cài đặt AVLTree insert/remove`, `test: thêm test_hash`, `docs: cập nhật Design.pdf`.